TOPEET 迅为

www.topeetboard.com

「新一代 AIOT 设备平台的优秀典范」

超长供货周期　7 X 24 小时稳定运行

软硬件全开源　丰富接口和高扩展性

iTOP-RK3568

迅为电子团队编著

官方微信公众号　迅为商城

010-8527-0708

www.topeetboard.com

Beijing TOPEET Electronics Co.Ltd

更新记录

更新版本	修改内容
V1.0	初版

目录

更新记录 2
目录 3
版权声明 4
更多帮助 5
实战：第一篇 OpenHarmony 配置 HDF 驱动控制 LED 6
第 1 章 GPIO 基础知识 6
1.1 什么是 GPIO 6
1.2 GPIO 引脚计算 6
1.3 GPIO 用户态测试 7
第 2 章 内核 HDF 驱动框架架构 9
2.1 驱动架构 9
2.2 平台驱动 11
2.3 外设驱动 11
2.4 HCS 简介 11
第 3 章 实操-HDF 驱动配置 LED 15
3.1 查看原理图 15
3.2 修改 HCS 硬件配置 15
3.3 创建私有配置文件 16
3.4 新增 topeet 子系统 17
3.4.1 编写 bundle.json 文件 18
3.4.2 编写内核 LED HDF 驱动程序 20
3.4.3 接口函数 25
3.4.4 添加内核编译 30
3.4.5 编写应用 APP 30
3.5 在产品中新增子系统 35
3.6 编译源码 36
3.7 LED 测试 36
实战：第二篇 Openharmony 配置 HDF 控制 UART 39
第 4 章 UART 基础知识 39
4.1 什么是串口 39
4.2 开发板串口简介 39
第 5 章 UART 接口运作机制 40
第 6 章 实操-HDF 驱动配置 UART 41
6.1 修改 HCS 配置 41
6.2 配置 rk3568_uart_config.hcs 42
6.3 Openharmony UART 平台驱动 43
6.4 UART 应用开发 43
6.4.1 UART 驱动 API 接口介绍 43
6.4.2 编写应用测试 APP 45
6.5 编译源码 49
6.6 UART 测试 50

版权声明

本文档版权归北京迅为电子有限公司所有。未经本公司书面许可，任何单位和个人无权以任何形式复制、传播、转载本文档的任何内容，违者将被追究法律责任。

更多帮助

注意事项与维护

- 请注意和遵循标注在产品上的所有警示和指引信息；
- 请勿带电插拔核心板及外围模块；
- 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；
- 请使用配套电源适配器，以保证电压、电流的稳定；
- 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；
- 请保持产品干燥，如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干；
- 请勿使用有机溶剂或腐蚀性液体清洗本产品；
- 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；
- 如果在震动场景使用，请做好核心板与底板的固定，避免核心板跌落损坏；
- 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；
- 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；
- 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

资料的更新

为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群推送。

关注“迅为电子”微信公众号，不定期分享教程、资料和行业干货及产品一线资料。

迅为新媒体账号

官网：https://www.topeetboard.com

知乎 https://www.zhihu.com/people/topeetabc123

CSDN：https://blog.csdn.net/BeiJingXunWei

售后服务政策

1. 如产品使用过程中出现硬件故障可根据售后服务政策进行维修
2. 服务政策：参见官方网售后服务说明 https://www.topeetboard.com/sydymfl/Service/bx.html

送修地址：

1. 地址：北京市海淀区永翔北路 9 号中国航发大厦三层
2. 联系人：迅为开发板售后服务部
3. 电话：010-85270716
4. 邮编：100094
5. 邮寄须知：建议使用顺丰、圆通或韵达，且不接受任何到付

技术支持范围

1. 了解产品的软、硬件资源提供情况咨询
2. 产品的软、硬件手册使用过程中遇到的问题
3. 下载和烧写更新系统过程中遇到的问题
4. 产品用户的资料丢失、更新后重新获取
5. 产品的故障判断及售后维修服务。

PS：（由于嵌入式系统知识范围广泛，我们无法保证对各种问题都能一一解答，部分内容无法供技术支持，只能提供建议。）

技术支持

1. 周一至周五：（法定节假日除外）

上午 9:00 ~ 11:30 / 下午 13:30 ~ 17:30

2. QQ 技术交流群： 824412014
822183461
95631883
861311530

实战：第一篇 OpenHarmony 配置 HDF 驱动控制 LED

第 1 章 GPIO 基础知识

1.1 什么是 GPIO

GPIO 意为通用输入/输出接口，GPIO 端口可以配置为输入或输出状态，用于产生或读取高低电平信号。其高低电平的数量、波形组合及变化方式没有任何协议限制，用户可以根据需求自由设定。GPIO 广泛应用于驱动简单外设，如 LED、按键、开关等，不需要复杂的协议支持。

与 GPIO 不同，专用 IO 端口通常是为特定通信协议设计的，具有更严格的协议要求。它们的高低电平数量、波形组合以及波形的持续时间必须严格遵循相应协议的规范。常见的专用 IO 协议包括 I2C、SPI、UART 和 PWM 等。这些接口通常用于数据传输或信号控制，如在传感器、显示模块、通信设备等应用中，要求精确的时序和信号格式。

芯片通常需要提供丰富的功能和外部接口，以满足各种应用需求。然而，由于芯片的管脚（pin）数量有限，为了最大化资源的利用，许多 IO 管脚支持多功能设计，并通过软件配置实现管脚的分时复用。也就是说，同一个管脚可以根据需求在不同的功能模式之间切换。需要注意的是，并非所有 IO 管脚都可以用作 GPIO。某些管脚被专门设计用于支持特定的专用接口（例如用于连接外部存储芯片或通信模块），这些管脚的功能是固定的，无法配置为 GPIO。而另一些管脚则可以自由配置为 GPIO，供用户根据实际需求灵活使用。

1.2 GPIO 引脚计算

为了更高效地管理数量众多的 GPIO，通常会将 GPIO 管脚按照一定规则进行分组管理。每个 GPIO 管脚都会被分配一个组号和组内偏移（offset）值，后者表示该管脚在所在组中的位置。通过这种分组方式，可以简化 GPIO 的访问和配置。

不同的芯片，其 GPIO 的分组数量和每组内包含的管脚数量可能有所不同。这些设计差异通常取决于芯片的架构和应用需求。例如，在 RK3568 芯片中有 5 组 GPIO bank：GPIO0~GPIO4，每组又以 A0~A7, B0~B7, C0~C7, D0~D7 作为编号区分,常用以下公式计算引脚：

```
GPIO pin 脚计算公式：pin = bank * 32 + number        //bank 为组号，number 为小组编号
GPIO 小组编号计算公式：number = group * 8 + X
```

引脚编号=控制寄存器的寄存器基数+控制引脚寄存器位数。在 rk3568 中，GPIO_number 的计算方法为： n*32 + (K-A)*8 + x; 括号里面的 A、 B、 C、 D 分别代表数值 0、 1、 2、 3，在计算时候分别对应即可。

下面演示 LED9 用户 LED 灯的 GPIO0_PB7 pin 脚计算方法：

```
bank = 0;        //GPIO0_B7=> 0, bank ∈ [0,4]
group = 1;       //GPIO0_B7 => 1, group ∈ {(A=0), (B=1), (C=2), (D=3)}
X = 7;          //GPIO4_D7 => 5, X ∈ [0,7]
number = group * 8 + X = 1 * 8 + 7 =15
pin = bank*32 + number= 0 * 32 + 15 = 15;
```

1.3 GPIO 用户态测试

sysfs 控制接口为/sys/class/gpio/export 和/sys/class/gpio/unexport。如下图所示：

```
[root@RK356X:/]# cd /sys/class/gpio
[root@RK356X:/sys/class/gpio]# ls
export   gpiochip0    gpiochip32   gpiochip64  unexport
gpio114  gpiochip128  gpiochip511  gpiochip96
[root@RK356X:/sys/class/gpio]#
```

/sys/class/gpio/export 用于将 GPIO 控制从内核空间导出到用户空间。/sys/class/gpio/unexport 用于取消 GPIO 控制从内核空间到用户空间的导出。 export 和 unexport，他们都是只写的。GpiochipX 代表 GPIO 控制器。

export：用于将指定编号的 GPIO 引脚导出。在使用 GPIO 引脚之前，需要将其导出，导出成功之后才能使用它。注意 export 文件是只写文件，不能读取，将一个指定的编号写入到 export 文件中即可将对应的 GPIO 引脚导出，以 GPIO0_PB7 为例（pin 计算值为 15）使用 export 文件进行导出，导出成功如下图所示：

```
echo 15 > export
```

```
[root@RK356X:/sys/class/gpio]# echo 15 > export
[root@RK356X:/sys/class/gpio]# ls
export   gpiochip0    gpiochip32   gpiochip64  unexport
gpio15   gpiochip128  gpiochip511  gpiochip96
[root@RK356X:/sys/class/gpio]#
```

会发现在/sys/class/gpio 目录下生成了一个名为 gpio15 的文件夹（gpioX，X 表示对应的编 号），该文件夹就是导出来的 GPIO 引脚对应的文件夹，用于管理、控制该 GPIO 引脚。

unexport：将导出的 GPIO 引脚删除。当使用完 GPIO 引脚之后，需要将导出的引脚删除，同样该文件也是只写文件、不可读，使用 unexport 文件进行删除 GPIO0_PB7，删除成功如下图所示：

```
echo 15 > unexport
```

```
[root@RK356X:/sys/class/gpio]# ls
export   gpiochip0    gpiochip32   gpiochip64  unexport
gpio15   gpiochip128  gpiochip511  gpiochip96
[root@RK356X:/sys/class/gpio]# echo 15 > unexport
[root@RK356X:/sys/class/gpio]# ls
export      gpiochip128  gpiochip511  gpiochip96
gpiochip0   gpiochip32   gpiochip64   unexport
[root@RK356X:/sys/class/gpio]#
```

可以看到之前生成的 gpio15 文件夹就会消失！

需要注意的是，并不是所有 GPIO 引脚都可以成功导出，如果对应的 GPIO 已经被导出或者在内核中被使用了，那便无法成功导出，导出失败如下图所示：

```
[root@RK356X:/sys/class/gpio]# echo 15 > export
sh: write error: Device or resource busy
[root@RK356X:/sys/class/gpio]#
```

出现上图报错的原因是该 GPIO 已经被其他 GPIO 使用，需要在内核中找到使用 GPIO 的驱动，并取消该驱动才可以正常使用 GPIO。在使用 GPIO15 时，需要取消 Linux 内核源码中 LED 灯的配置，如下所示：

```
leds: leds {
    compatible = "gpio-leds";
    work_led: work {
        lable = "user1";
        gpios = <&gpio0 RK_PB7 GPIO_ACTIVE_HIGH>;
        linux,default-trigger = "heartbeat";
        default-state = "on";
    };
};
```

再次使用以下命令导出 GPIO0_PB7 引脚，导出成功之后进入 gpio15 文件夹如下图所示：

```
echo 15 > export
```

```
active_low  device  direction  edge  power  subsystem  uevent  value
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
```

可以看到 gpio15 文件夹下分别有 active_low、device、direction、edge、power、subsystem、uevent、value 八个文件，需要关心的文件是 active_low、direction、edge 以及 value 这四个属性文件，接下来分别介绍这四个属性文件的作用：

direction：配置 GPIO 引脚为输入或输出模式。该文件可读、可写，读表示查看 GPIO 当前是输入还是输出模式，写表示将 GPIO 配置为输入或输出模式；读取或写入操作可取的值为"out"（输出模式）和"in"（输入模式）。

在“/sys/class/gpio/gpio15”目录下使用 cat 命令查看 direction 输入输出模式，如下图所示：

```
cat direction
```

```
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]# cat dir
ection
in
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
```

默认状态下的输入输出状态为“in”,由于 direction 为可读可写，可以使用以下命令将模式配置为输出，配置完成如下图所示

```
echo out > direction
cat direction
```

```
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]# echo ou
t > direction
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]# cat dir
ection
out
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
```

active_low：用于控制极性得属性文件，可读可写，默认情况下为 0，使用 cat 命令进行文件内容的查看，如下图所示 ：

```
cat active_low
```

```
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]# cat act
ive_low
0
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
```

当 active_low 等于 0 时， value 值若为 1 则引脚输出高电平，value 值若为 0 则引脚输出低电平。当 active_low 等于 1 时 ，value 值若为 0 则引脚输出高电平，value 值若为 1 则引脚输出低电平。

edge：控制中断的触发模式，该文件可读可写。在配置 GPIO 引脚的中断触发模式之前，需将其设置为输入模式，四种触发模式的设置如下所示：

1	非中断引脚：echo "none" > edge
2	上升沿触发：echo "rising" > edge
3	下降沿触发：echo "falling" > edge
4	边沿触发： echo "both" > edge

value: 设置高低电平，如果我们要把这个管脚设置成高电平，我们只需要给 value 设置成 1 即可，反之，则设置成 0。使用命令

```
echo 1 > value
```

反之，把 GPIO 设置成低电平，使用命令

```
echo 0 > value
```

```
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]# echo 1
>value
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]# echo 0
>value
[root@RK356X:/sys/devices/platform/fdd60000.gpio/gpiochip0/gpio/gpio15]#
```

第 2 章 内核 HDF 驱动框架架构

本篇实战我们将要配置 HDF 驱动控制 LED，在配置之前，我们首先要明白什么是 HDF，以及 HDF 驱动开发的相关概念。

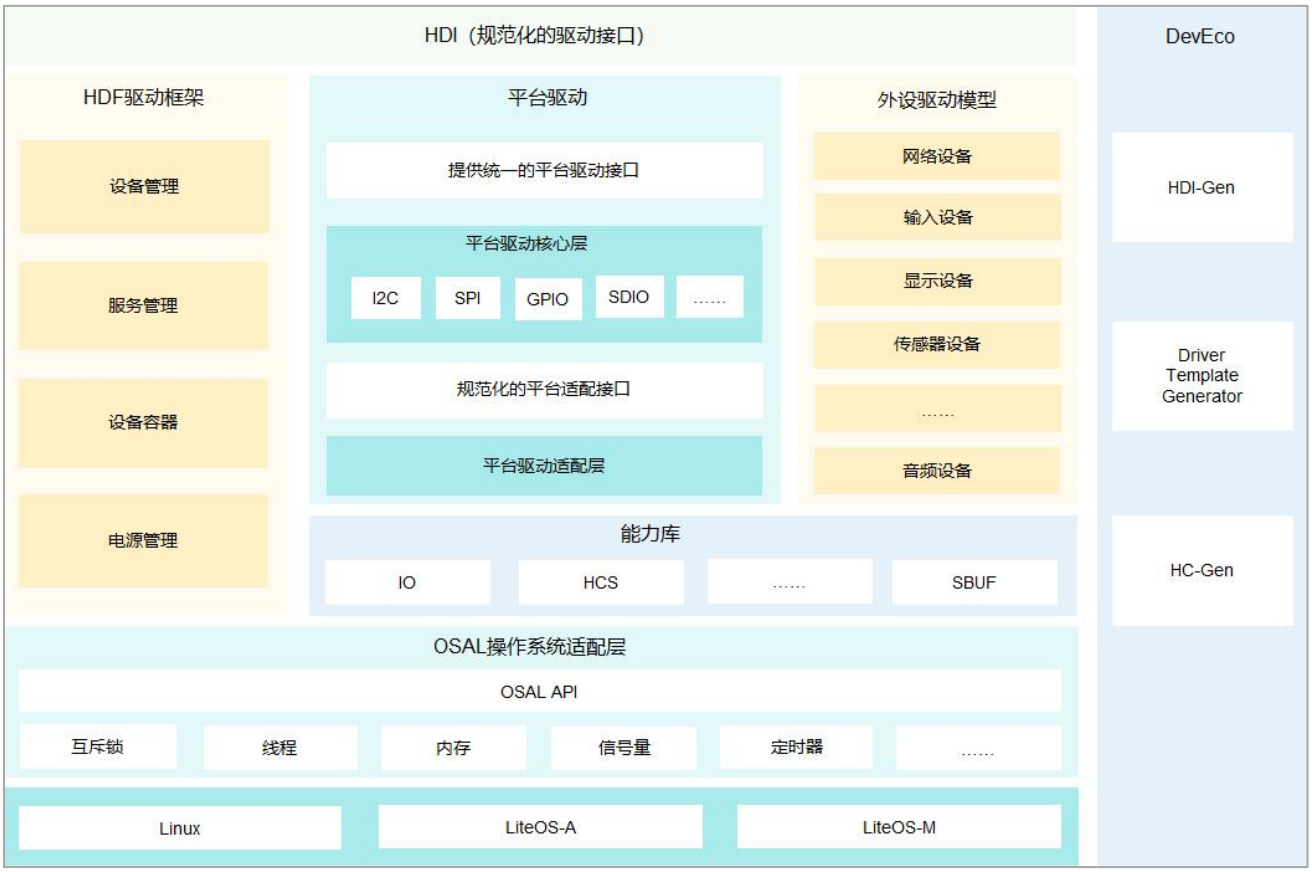
2.1 驱动架构

下面的视频——OpenHarmony 功能框架

https://www.bilibili.com/video/BV18t421t7F6?spm_id_from=333.788.videopod.episodes&vd_source=5db6c77875eccc47b243c4bfa15616d6&p=3 中提到了 HDF(Hardware Driver Foundation)。接下来我们了解一下什么是 HDF。

HDF(Hardware Driver foundation)驱动框架，为驱动开发者提供驱动框架能力，包括驱动加载，驱动服务管理和驱动消息机制。旨在构建统一的驱动架构平台，为驱动开发者提供更精准，更高效的开发环境，力求做到一次开发，多系统部署。

HDF 驱动架构采用 C 语言面向对象编程模型构建，通过平台解耦，内核解耦，来达到兼容不同内核，统一平台底座的目的。HDF 驱动框架架构如图所示：



- HDF 驱动架构主要组成部分：
- HDI（Hardware Device Interface，硬件设备统一接口）层：通过规范化的设备接口标准，为系统提供统一、稳定的硬件设备操作接口。
- HDF 驱动框架：提供统一的硬件资源管理、驱动加载管理、设备节点管理、设备电源管理以及驱动服务模型等功能，需要包含设备管理、服务管理、DeviceHost、PnPManager 等模块。
- 统一的配置界面：支持硬件资源的抽象描述，屏蔽硬件差异，可以支撑开发者开发出与配置信息不绑定的通用驱动代码，提升开发及迁移效率，并可通过 HC-Gen 等工具快捷生成配置文件。
- 操作系统抽象层（OSAL，Operating System Abstraction Layer）：提供统一封装的内核操作

相关接口，屏蔽不同系统操作差异，包含内存、锁、线程、信号量等接口。

- 平台驱动：为外设驱动提供 Board 硬件（如：I2C/SPI/UART 总线等平台资源）操作统一接口，同时对 Board 硬件操作进行统一的适配接口抽象以便于不同平台迁移。
- 外设驱动模型：面向外设驱动，提供常见的驱动抽象模型，主要达成两个目的，提供标准化的器件驱动，开发者无需独立开发，通过配置即可完成驱动的部署；提供驱动模型抽象，屏蔽驱动与不同系统组件间的交互，使得驱动更具备通用性。

2.2 平台驱动

OpenHarmony 平台驱动（Platform Driver）是为平台设备（Platform Device）提供驱动支持的接口，帮助系统和外设之间的互通。这里的平台设备包括总线接口（如 I2C、UART）以及特定硬件资源（如 GPIO、RTC 等）。平台驱动框架是 OpenHarmony 驱动框架的核心部分，基于 HDF 驱动框架、操作系统适配层和驱动配置管理，提供一个统一的标准模型，帮助开发者实现各类平台设备驱动。

平台驱动框架的主要特点包括：

- 统一的平台设备访问接口：为平台设备操作提供统一的接口，屏蔽不同硬件平台（SoC）和操作系统（OS）的差异。
- 统一的适配接口：为平台设备驱动提供统一的接口，让开发者只需关注硬件控制，而无需关心设备管理或其他公共流程。
- 提供公共能力：提供设备注册、管理、访问控制等功能，且与特定 SoC 平台无关。

平台驱动框架支持的设备类型包括但不限于：ADC、DAC、GPIO、HDMI、I2C、I3C、MIPI_CSI、MIPI_DSI、MMC、Pin、PWM、Regulator、RTC、SDIO、SPI、UART、WatchDog 等。

2.3 外设驱动

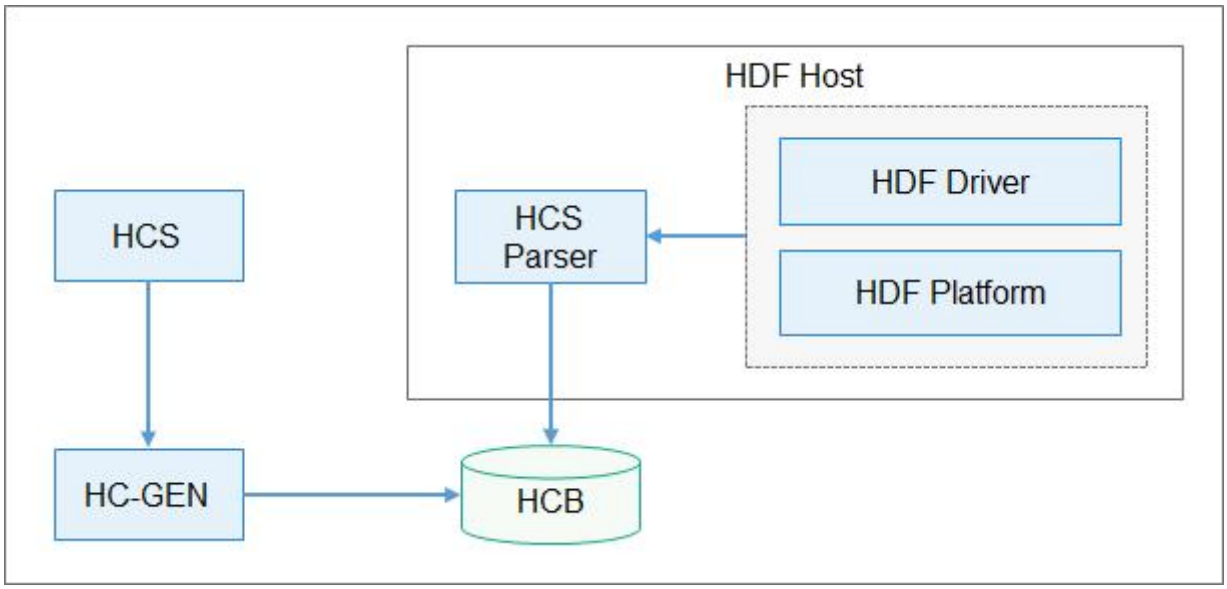
OpenHarmony 在 HDF 驱动框架和平台驱动框架的基础上，为外设设备提供了标准化的驱动模型。这些驱动模型可以帮助开发者减少重复开发工作，并提供统一的抽象层，屏蔽了驱动与系统其他组件之间的复杂交互，使得驱动更加通用、易于移植。

目前，OpenHarmony 支持的外设设备类型包括但不限于：音频（Audio）、摄像头（Camera）、编解码器（Codec）、人脸识别（Face_auth）、指纹识别（Fingerprint_auth）、LCD 屏幕（LCD）、灯光控制（Light）、运动传感器（Motion）、引脚认证（Pin_auth）、传感器（Sensor）、触摸屏（Touchscreen）、USB、用户认证（User_auth）、振动器（Vibrator）、无线局域网（WLAN）等。

2.4 HCS 简介

HCS(HDF Configuration Source)是 HDF 驱动框架的配置描述源码，内容以 Key-Value 为主要形式。它实现了配置代码与驱动代码解耦，便于开发者进行配置管理。HC-GEN(HDF Configuration Generator) 是 HCS 配置转换工具，可以将 HDF 配置文件转换为软件可读取的文件格式。在高性能环境中，HCS 经过 HC-GEN 编译生成 HCB（HDF Configuration Binary）二进制文件，HDF 驱

动框架中的 HCS Parser 模块会从 HCB 文件中重建配置树，HDF 驱动模块使用 HCS Parser 提供的配置读取接口获取配置内容。HCB 配置使用流程图如下图所示：



接下来介绍一下 HCS 的语法，HCS 配置语法保留了以下关键字。

（1）保留关键字

关键字	用途	说明
root	配置根节点	-
include	引用其他 HCS 配置文件	-
delete	删除节点或属性	只能用于操作 include 导入的配置树
template	定义模板节点	-
match_attr	用于标记节点的匹配查找属性	解析配置时可以使用该属性的值查找到对应节点

（2）基本结构

HCS 主要分为属性（Attribute）和节点（Node）两种结构。

属性

属性是最小的配置单元，表示一个独立的配置项，语法格式如下：

```
attribute_name = value;
```

- attribute_name：由字母、数字、下划线组成，且必须以字母或下划线开头，字母区分大小写。
- value：可以是以下几种类型：
  数字常量：支持二进制、八进制、十进制、十六进制，具体类型参考数据类型部分。
  字符串：使用双引号（""）括起来。
  节点引用：可以引用其他节点

每个属性必须以分号（;）结束，并且必须属于某个节点（Node）。

（3）节点

节点是由一组属性组成的集合，语法格式如下：

```
node_name {
    module = "sample";
    ...
```

```
}
```

node_name：由字母、数字、下划线组成，必须以字母或下划线开头，字母区分大小写。节点的内容用大括号（{}）包围，且不需要添加结束符号。
root 是保留关键字，用于声明配置表的根节点。每个配置表必须以 root 节点开始。
root 节点中必须包含 module 属性，用于标识配置所属的模块。

（4）数据类型

在 HCS 中，属性的类型自动推断，不需要显式指定。支持的数据类型如下：

- 整型：根据数据长度自动选择合适的类型，支持二进制、八进制、十进制、十六进制表示。
- 字符串：使用双引号（""）括起来。
- 数组：支持整型和字符串数组，但不支持混合类型。整型数组中的 uint32_t 与 uint64_t 会向 uint64_t 数组转换。
- 布尔类型（bool）：true 表示真，false 表示假。

（5）预处理

include：用于导入其他 HCS 文件，语法格式如下：

```
#include "foo.hcs"
```

文件名使用双引号（""），并且可以使用相对路径。多个 include 文件时，如果存在相同的节点，后导入的文件会覆盖前一个。

（6）注释

注释：支持单行注释和多行注释。

```
单行注释：// comment
多行注释：/* comment */（不支持嵌套）
```

（7）引用修改

引用修改（Reference Modification）：引用修改允许在当前节点中修改其他节点的内容，语法为：

```
node :& source_node
```

（8）节点复制

节点复制可以实现在节点定义时从另一个节点先复制内容，用于定义内容相似的节点。语法为：

```
node : source_node
```

（9）删除

要对 include 导入的 base 配置树中不需要的节点或属性进行删除，可以使用 delete 关键字。举个例子，sample1.hcs 通过 include 导入了 sample2.hcs 中的配置内容，并使用 delete 删除了 sample2.hcs 中的 attribute2 属性和 foo_2 节点

```
// sample2.hcs
root {
    attr_1 = 0x1;
    attr_2 = 0x2;
    foo_2 {
        t = 0x1;
    }
}
```

```
// sample1.hcs
#include "sample2.hcs"
root {
    attr_2 = delete;
    foo_2 : delete {
    }
}
```

（10）属性引用

为了在解析配置时快速定位到关联的节点，可以把节点作为属性的右值，通过读取属性查找到对应节点。语法为

```
attribute = &node;
```

上面语句表示 attribute 的值是一个节点 node 的引用，在解析时可以用这个 attribute 快速定位到 node，便于关联和查询其他 node。

（11）模板

模板的用途在于生成严格一致的 node 结构，以便对同类型 node 进行遍历和管理。使用 template 关键字定义模板 node，子 node 通过双冒号“::”声明继承关系。子节点可以改写或新增但不能删除 template 中的属性，子节点中没有定义的属性将使用 template 中的定义作为默认值。示例如下：

```
root {
    module = "sample";
    template foo {
        attr_1 = 0x1;
        attr_2 = 0x2;
    }

    bar :: foo {
    }

    bar_1 :: foo {
        attr_1 = 0x2;
    }
}
```

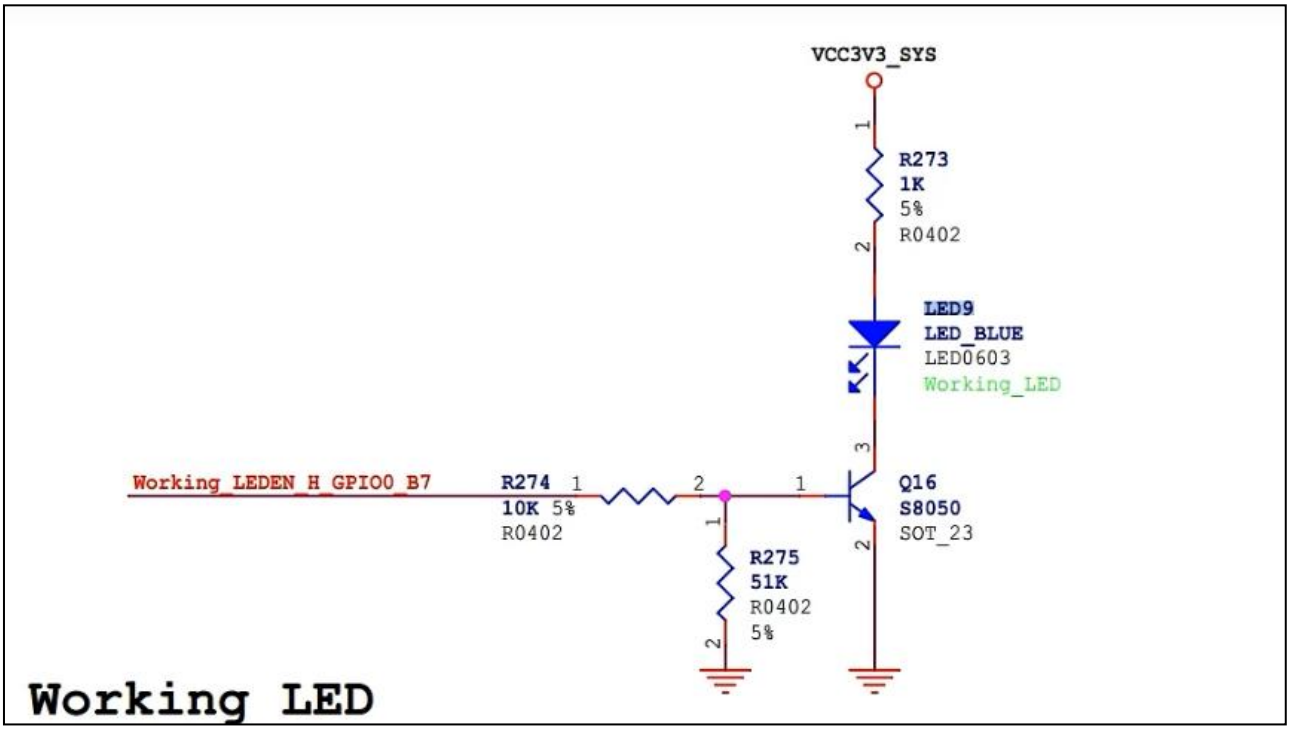
在上面示例中，bar 和 bar_1 节点继承了 foo 节点，生成配置树节点结构与 foo 保持了完全一致，只是属性的值不同。

第 3 章 实操-HDF 驱动配置 LED

从本章节开始，我们来实操一下，配置 HDF 驱动控制 LED。

3.1 查看原理图

首先打开底板原理图，如下图所示：



由上图可以看出，LED 灯是由 GPIO0_B7 控制的。当 GPIO0_B7 为高电平时，三极管 Q16 导通，LED9 点亮。当 GPIO0_B7 为低电平时，三极管 Q16 截止，LED9 不亮。由 1.2 小节可以计算出 GPIO 的引脚编号是 15。

3.2 修改 HCS 硬件配置

驱动的设备描述修改/vendor/hihope/rk3568/hdf_config/khdf/device_info/device_info.hcs 文件，添加如下代码，如下所示：

```
device_topeet_led :: device {
    device0::deviceNode {
        policy = 2;
        priority = 100;
        preload = 0;
        permission = 0666;
        moduleName = "topeet_led_driver";
        serviceName = "topeet_led_service";
        deviceMatchAttr = "topeet_led_config";
```

```
10
11          }
12  }
```

```
platform :: host {
    hostName = "platform_host";
    priority = 50;

    device_topeet_led :: device {
        device0::deviceNode {
            policy = 2;
            priority = 100;
            preload = 0;
            permission = 0666;
            moduleName = "topeet_led_driver";
            serviceName = "topeet_led_service";
            deviceMatchAttr = "topeet_led_config";

        }
    }

    device_gpio :: device {
        device0 :: deviceNode {
            policy = 2;
```

接下来解释一下上面的节点配置

- device_topeet_led 设备节点归类于 platform 这个 host
- device_topeet_led :: device 表示 led 类设备
- device0::deviceNode 表示 led 类设备下的某个具体设备节点的配置
- policy = 2;表示驱动服务发布策略，内核态用户态都可调用
- priority = 100;表示驱动启动优先级
- preload = 0;表示驱动按需加载字段，启动加载
- permission = 0666;表示驱动创建设备节点
- moduleName = "topeet_led_driver";表示驱动名称是 topeet_led_driver，必须和驱动入口结构中的 moduleName 值一致。
- serviceName = "topeet_led_service";表示驱动对外发布服务的名称，必须唯一
- deviceMatchAttr = "topeet_led_config";表示驱动私有数据匹配关键词，必须和驱动私有数据配置节点的 match_attr 匹配

3.3 创建私有配置文件

接下来新建 vendor/hihope/rk3568/hdf_config/khdf/topeet/topeet_config.hcs 文件，topeet_config.hcs 为驱动私有配置文件，用来填写一些驱动的默认配置信息。HDF 框架在加载驱动时，会获取相应的配置信息并将其保存在 HdfDeviceObject 的 property 中。这些配置信息通过 Bind 和 Init 方法传递给驱动。

topeet_config.hcs 具体内容如下所示：

```
    root {
        platform{
            topeet_led_config {
    //该字段的值必须和 device_info.hcs 中的 deviceMatchAttr 一致

                match_attr = "topeet_led_config";
                led_version = 1;//版本号
                led_number = 15;//GPIO 引脚号
            }
        }
}
```

驱动私有配置文件写完之后，我们需要将该配置文件添加到板级配置入口文件 vendor/hihope/rk3568/hdf_config/khdf/hdf.hcs 中，如下图所示：

```
#include "audio/dma_config.hcs"
#include "audio/dsp_config.hcs"
#include "audio/analog_headset_config.hcs"
#include "light/light_config.hcs"
#include "vibrator/vibrator_config.hcs"
#include "vibrator/linear_vibrator_config.hcs"
#include "vibrator/drv2605l_linear_vibrator_config.hcs"
#include "lcd/lcd_config.hcs"
#include "topeet/topeet_config.hcs"
root {
    module = "rockchip,rk3568_chip";
}
```

3.4 新增 topeet 子系统

在 Openharmony 源码根目录下新建 topeet 文件夹及其文件夹下的文件。目录如下所示：

```
topeet@ubuntu:~/OH4.1/OpenHarmony-v4.1-Release/OpenHarmony/topeet$ tree
.
├── bundle.json
├── demos
│   └── hdf_led
│       ├── app
│       │   ├── BUILD.gn
│       │   └── led_test.c
│       └── driver
│           ├── led_driver.c
│           └── Makefile
└── topeet_config.hcs

4 directories, 6 files
```

接下来依次解释一下每个文件的作用。

bundle.json：

demos：组件目录

hdf_led：子组件目录

app：led 应用层目录

├── BUILD.gn：应用 APP 的 GN 文件

└── led_test.c：应用层 LED 测试程序

driver：内核 HDF 驱动程序目录

├── led_driver.c：内核 LED HDF 驱动程序

└── Makefile：内核 LED HDF 驱动编译脚本

3.4.1 编写 bundle.json 文件

bundle.json 文件内容如下所示：

```
1  {
2          "name":"@ohos/demos",
3          "description":"topeet demos",
4          "version":"4.1",
5          "license":"Apache-2.0",
6          "publishAs":"code-segment",
7          "segment":{
8              "destPath":"topeet/demos"
9          },
10         "dirs":{},
11         "scripts":{},
12         "component":{
13             "name":"demos",
14             "subsystem":"topeet",
15             "features":[],
16             "syscap":[],
17             "adapted_system_type":["standard"],
```

```
18                  "rom":"675KB",
19                  "ram":"7400KB",
20                  "deps":{
21                      "components":[
22                          "c_utils",
23                          "hilog",
24                          "hdf_core",
25                          "napi"
26                      ],
27                      "third_party":[]
28                  },
29                  "build":{
30                      "sub_component":[
31                          "//topeet/demos/hdf_led/app:led_test"
32                      ]
33
34                  }
35              }
36  }
```

下面是对各个字段的解释：

1. name: "@ohos/demos" - 这是组件或项目的名称，这里表示它属于 OHOS（OpenHarmony OS）生态系统下的一个名为"demos"的组件。
2. description: "topeet demos" -它描述了组件的简短说明
3. version: "4.1" - 组件的版本号。
4. license: "Apache-2.0" - 组件使用的许可证类型，这里是 Apache 2.0 许可证。
5. publishAs: "code-segment" - 表示这个组件或项目是以代码段的形式发布的。
6. segment:

◆ destPath: "topeet/demos" - 代码段的目标路径，即这个组件或项目在系统中的存放位置。

7. dirs: {} - 一个空对象，可能用于定义与组件相关的目录结构，但在这个配置中未使用。
8. scripts: {} - 一个空对象，可能用于定义与组件相关的脚本，如构建脚本、测试脚本等，但在这个配置中未使用。
9. component:

◆ name: "demos" - 组件的名称。

◆ subsystem: "topeet" - 组件所属的子系统名称。

◆ features: [] - 组件的功能列表，这里为空，表示没有列出特定功能。

- syscap: [] - 系统能力列表，这里为空，表示没有列出特定的系统能力。
- adapted_system_type: ["standard"] - 适配的系统类型，这里表示适用于标准系统。
- rom: "675KB" - 组件所需的 ROM 大小。
- ram: "7400KB" - 组件所需的 RAM 大小。
- deps:

  components: ["c_utils", "hilog", "hdf_core", "napi"] - 组件依赖的其他组件列表。

  third_party: [] - 第三方依赖列表，这里为空。
- build:

sub_component: ["//topeet/demos/hdf_led/app:led_test"] - 构建时包含的子组件路径，这里指定了一个具体的构建目标。

这个 JSON 配置文件提供了关于如何构建、部署和管理这个名为"demos"的组件的详细信息。它定义了组件的基本属性、依赖关系、构建信息以及目标系统类型等。

3.4.2 编写内核 LED HDF 驱动程序

接下来编译 LED 驱动，该驱动用于在基于华为设备框架（HDF）的系统中控制 LED 灯的开关，完整代码如下所示：

```
#include "device_resource_if.h"
#include "hdf_device_desc.h"
#include "hdf_log.h"
#include "gpio_if.h"

#define HDF_LOG_TAG led_driver
#define LED_WRITE 1
#define LED_VERSION 1
#define LED_ON 1
#define LED_OFF 0

struct Led_config{
    uint32_t led_version;
    uint32_t led_number;
};

struct Led_config g_LedCfg = {0};

/**
 * @brief 控制 LED 的 GPIO 引脚
```

```
21      *
22      * 根据传入的 GPIO 引脚号和模式，控制 LED 的开关状态。
23      *
24      * @param gpio GPIO 引脚号
25      * @param mode 控制模式，LED_ON 表示打开 LED，LED_OFF 表示关闭 LED
26      *
27      * @return 成功返回 HDF_SUCCESS，失败返回 HDF_FAILURE
28      */
29     static int32_t LedGpioCtl(uint16_t gpio, uint32_t mode){
30         // 设置 GPIO 电平为高电平
31         uint16_t level = GPIO_VAL_HIGH;
32
33         // 设置 GPIO 为输出方向
34         if(HDF_SUCCESS != GpioSetDir(gpio, GPIO_DIR_OUT)){
35             // 设置 GPIO 方向失败
36             HDF_LOGE("GpioSetDir fail");
37             return HDF_FAILURE;
38         }
39
40         // 根据 mode 设置 GPIO 电平
41         if(mode == LED_ON){
42             level = GPIO_VAL_HIGH;
43         }else if(mode==LED_OFF){
44             level = GPIO_VAL_LOW;
45         }
46
47         // 日志记录 GPIO 操作
48         HDF_LOGE("%s:Write gpio %d:%d",__func__,gpio,mode);
49
50         // 向 GPIO 写入电平
51         if(HDF_SUCCESS != GpioWrite(gpio, level)){
52             // 写入 GPIO 电平失败
53             HDF_LOGE("GpioWrite fail",__func__);
54         }
55
56         return HDF_SUCCESS;
57     }
58
59
60     /**
61      * @brief 驱动 LED 设备
62      *
63      * 根据传入的命令 ID 和数据，控制 LED 设备的状态。
64      *
```

```
65          * @param client HDF 设备客户端指针
66          * @param cmdId  命令 ID，用于指示执行的操作类型
67          * @param dataBuf  输入数据缓冲区指针，包含需要传递给设备的数据
68          * @param replyBuf  输出数据缓冲区指针，用于存储设备返回的数据
69          *
70          * @return  返回操作结果，成功返回 HDF_SUCCESS，失败返回相应的错误码
71          */
72        int32_t LedDriverDispatch(struct HdfDeviceIoClient *client, int32_t cmdId, struct HdfSBuf *dataBuf,
73  struct HdfSBuf *replyBuf){
74
75              int32_t result = HDF_FAILURE;
76              int32_t LedMode = 0;
77
78              // 检查客户端和设备是否为空
79              if(client == NULL || client->device == NULL){
80                  HDF_LOGE("driver device is NULL");
81                  return HDF_ERR_INVALID_OBJECT;
82              }
83
84              // 检查 LED 配置版本是否支持
85              if(g_LedCfg.led_version != LED_VERSION){
86                  HDF_LOGE("led version is not support");
87                  return HDF_FAILURE;
88              }
89
90              switch(cmdId){
91                  case LED_WRITE:
92                      // 从数据缓冲区读取 LED 模式
93                      result = HdfSbufReadInt32(dataBuf,&LedMode);
94                      if(result ){
95                          // 根据 LED 模式控制 GPIO
96                          LedGpioCtl(g_LedCfg.led_number, (LedMode == LED_ON) ? LED_ON: LED_OFF);
97                      }
98                      break;
99                  default:
100                     // 不支持的命令 ID
101                     HDF_LOGE("cmdId is not support");
102                     break;
103             }
104             return result;
105
106       }
107
108       /**
```

```
109         * @brief 绑定 LED 设备驱动
110         *
111         * 将 LED 设备驱动绑定到 HDF 设备对象上。
112         *
113         * @param deviceObject HDF 设备对象指针
114         *
115         * @return 返回 HDF 状态码，成功返回 HDF_SUCCESS，失败返回相应的错误码
116         */
117        int32_t HdfLedDriverBind(struct HdfDeviceObject *deviceObject){
118            // 检查 deviceObject 是否为空
119            if(deviceObject == NULL){
120                // 如果为空，则记录错误日志并返回错误码
121                HDF_LOGE("HdfLedDriverBind: %s failed",__func__);
122                return HDF_ERR_INVALID_OBJECT;
123            }
124
125            // 定义一个静态的 IDeviceIoService 结构体变量 ledDriverServ
126            static struct IDeviceIoService ledDriverServ = {
127                .Dispatch = LedDriverDispatch,
128            };
129
130            // 将 ledDriverServ 的地址赋值给 deviceObject 的 service 成员
131            deviceObject->service =(struct IDeviceIoService *)(&ledDriverServ);
132
133            // 记录绑定成功的日志，包括设备名称
134            HDF_LOGI("g_LedDriverEntry: %s success NodeName[%s]", __func__,
135 deviceObject->property->name);
136
137            // 返回成功码
138            return HDF_SUCCESS;
139        }
140
141
142        /**
143         * @brief 初始化 HDF LED 驱动
144         *
145         * 该函数用于初始化 HDF LED 驱动，从 HCS 配置文件中读取硬件相关配置属性。
146         *
147         * @param deviceObject 设备对象指针
148         *
149         * @return 初始化结果
150         *         - HDF_SUCCESS: 初始化成功
151         *         - HDF_ERR_INVALID_OBJECT: 设备对象无效
152         *         - HDF_FAILURE: 初始化失败
```

```
 */
int32_t HdfLedDriverInit(struct HdfDeviceObject *deviceObject){
    // 检查 deviceObject 是否为空
    if(deviceObject == NULL){
        HDF_LOGE("g_LedDriverEntry: %s failed",__func__);
        return HDF_ERR_INVALID_OBJECT;
    }

    // 获取 DeviceResourceIface 实例
    struct DeviceResourceIface *cfgops= NULL;
    cfgops = DeviceResourceGetIfaceInstance(HDF_CONFIG_SOURCE);

    // 检查 cfgops 及其方法 GetUint32 是否有效
    if (cfgops == NULL || cfgops->GetUint32 == NULL) {
        HDF_LOGE("%s:DeviceResourceGetIfaceInstance failed", __func__);
        return HDF_FAILURE;
    }

    // 读取 hcs 配置中的硬件相关配置属性：led_version
    // 读取 led_version
    if(cfgops->GetUint32(deviceObject->property,"led_version",&g_LedCfg.led_version,0)!=
HDF_SUCCESS){
        HDF_LOGE("%s: read led_version failed", __func__);
        return HDF_FAILURE;
    }

    // 读取引脚号：led_number
    // 读取 led_number
    if(cfgops->GetUint32(deviceObject->property,"led_number",&g_LedCfg.led_number,0)!=
HDF_SUCCESS){
        HDF_LOGE("%s:Gread led_number failed", __func__);
        return HDF_FAILURE;
    }

    // 打印初始化成功日志
    HDF_LOGI("g_LedDriverEntry: %s success", __func__);
    return HDF_SUCCESS;
}

/**
 * @brief 释放 HDF LED 驱动资源
 *
 * 该函数用于释放 HDF LED 驱动的资源。如果传入的 HdfDeviceObject 指针为空，则打印错误日
志并直接返回。
```

```
 * 如果 HdfDeviceObject 指针不为空，则打印成功日志并返回。
 *
 * @param HdfDeviceObject HDF 设备对象指针
 */
void HdfLedDriverRelease(struct HdfDeviceObject *HdfDeviceObject){
    // 如果传入的 HdfDeviceObject 为空
    if(HdfDeviceObject == NULL){
        // 打印错误日志
        HDF_LOGE("HdfLedDriverRelease: %s failed",__func__);
        // 返回
        return;
    }
    // 打印成功日志
    HDF_LOGI("HdfLedDriverRelease: %s success", __func__);
    // 返回
    return;
}



//定义了一个结构体 HdfDriverEntry 的实例 g_LedDriverEntry，并初始化了它的成员变量
struct HdfDriverEntry g_LedDriverEntry = {
    .moduleVersion = 1,
    .moduleName = "topeet_led_driver",
    .Bind = HdfLedDriverBind,
    .Init = HdfLedDriverInit,
    .Release = HdfLedDriverRelease,

};
//使用 HDF_INIT 宏来注册或初始化这个结构体实例 g_LedDriverEntry
HDF_INIT(g_LedDriverEntry);
```

3.4.3 接口函数

在一小节的代码中实现了一个简单的 LED 驱动，下面是对代码的详细解释：

包含的头文件如下所示：

```
#include "device_resource_if.h"：提供设备资源接口，用于从配置文件中读取设备信息
#include "hdf_device_desc.h"：包含 HDF 设备描述相关的定义
#include "hdf_log.h"：提供日志记录功能
#include "gpio_if.h：提供 GPIO 接口，用于控制 LED 灯的开关
```

宏定义如下所示：

```
#define HDF_LOG_TAG led_driver ：定义日志标签，用于区分不同模块的日志
```

```
#define LED_WRITE 1：定义 LED 控制命令的 ID
#define LED_VERSION 1: 定义 LED 驱动的版本号
#define LED_ON 1 ：定义 LED 灯打开的状态
#define LED_OFF 0：定义 LED 灯关闭的状态
```

数据结构如下所示：

```
struct Led_config{   //led_config 结构体用于存储 LED 配置信息，包括 LED 驱动版本号和 LED GPIO 编号
        uint32_t led_version;
        uint32_t led_number;
    };

    struct Led_config g_LedCfg = {0}; //全局变量，用于存储 LED 配置
```

g_LedDriverEntry 结构体是驱动入口结构体，如下所示，包含了驱动的版本号、模块名、绑定、初始化和释放函数。

```
struct HdfDriverEntry g_LedDriverEntry = {
        .moduleVersion = 1,
        .moduleName = "topeet_led_driver",
        .Bind = HdfLedDriverBind,
        .Init = HdfLedDriverInit,
        .Release = HdfLedDriverRelease,

    };
    HDF_INIT(g_LedDriverEntry);
```

HdfLedDriverInit 函数是驱动初始化函数。

- 参数：deviceObject（设备对象）。
- 流程：获取设备资源接口，读取设备配置中的 led_version 和 led_number（GPIO 号），并保存到全局配置变量中。

```
int32_t HdfLedDriverInit(struct HdfDeviceObject *deviceObject){
        // 检查 deviceObject 是否为空
        if(deviceObject == NULL){
            HDF_LOGE("g_LedDriverEntry: %s failed",__func__);
            return HDF_ERR_INVALID_OBJECT;
        }

        // 获取 DeviceResourceIface 实例
        struct DeviceResourceIface *cfgops= NULL;
        cfgops = DeviceResourceGetIfaceInstance(HDF_CONFIG_SOURCE);

        // 检查 cfgops 及其方法 GetUint32 是否有效
```

```
        if (cfgops == NULL || cfgops->GetUint32 == NULL) {
            HDF_LOGE("%s:DeviceResourceGetIfaceInstance failed", __func__);
            return HDF_FAILURE;
        }

        // 读取 hcs 配置中的硬件相关配置属性：led_version
        // 读取 led_version
        if(cfgops->GetUint32(deviceObject->property,"led_version",&g_LedCfg.led_version,0)!=
HDF_SUCCESS){
            HDF_LOGE("%s: read led_version failed", __func__);
            return HDF_FAILURE;
        }

        // 读取引脚号：led_number
        // 读取 led_number
        if(cfgops->GetUint32(deviceObject->property,"led_number",&g_LedCfg.led_number,0)!=
HDF_SUCCESS){
            HDF_LOGE("%s:Gread led_number failed", __func__);
            return HDF_FAILURE;
        }

        // 打印初始化成功日志
        HDF_LOGI("g_LedDriverEntry: %s success", __func__);
        return HDF_SUCCESS;
    }
```

HdfLedDriverRelease：驱动释放函数。

- 参数：HdfDeviceObject（设备对象）。
- 流程：记录日志，表示驱动释放成功。

```
    void HdfLedDriverRelease(struct HdfDeviceObject *HdfDeviceObject){
        // 如果传入的 HdfDeviceObject 为空
        if(HdfDeviceObject == NULL){
            // 打印错误日志
            HDF_LOGE("HdfLedDriverRelease: %s failed",__func__);
            // 返回
            return;
        }
        // 打印成功日志
        HDF_LOGI("HdfLedDriverRelease: %s success", __func__);
        // 返回
        return;
    }
```

HdfLedDriverBind：绑定解析函数

- 参数：deviceObject（设备对象）。
- 流程：将 LED 驱动的服务对象赋值给设备对象的服务成员。

```
int32_t HdfLedDriverBind(struct HdfDeviceObject *deviceObject){
    // 检查 deviceObject 是否为空
    if(deviceObject == NULL){
        // 如果为空，则记录错误日志并返回错误码
        HDF_LOGE("HdfLedDriverBind: %s failed",__func__);
        return HDF_ERR_INVALID_OBJECT;
    }

    // 定义一个静态的 IDeviceIoService 结构体变量 ledDriverServ
    static struct IDeviceIoService ledDriverServ = {
        .Dispatch = LedDriverDispatch,
    };

    // 将 ledDriverServ 的地址赋值给 deviceObject 的 service 成员
    deviceObject->service =(struct IDeviceIoService *)(&ledDriverServ);

    // 记录绑定成功的日志，包括设备名称
    HDF_LOGI("g_LedDriverEntry: %s success NodeName[%s]", __func__,
deviceObject->property->name);

    // 返回成功码
    return HDF_SUCCESS;
}
```

LedDriverDispatch：解析函数，解析应用层下发的命令，执行命令对应的操作，控制 led 灯的亮灭。

- 参数：client（客户端信息），cmdId（命令 ID），dataBuf（输入数据缓冲区），replyBuf（回复数据缓冲区）。
- 流程：检查设备对象的有效性，验证 LED 版本，根据命令 ID 读取数据并调用 LedGpioCtl 控制 LED。

```
int32_t LedDriverDispatch(struct HdfDeviceIoClient *client, int32_t cmdId, struct HdfSBuf *dataBuf,
struct HdfSBuf *replyBuf){

    int32_t result = HDF_FAILURE;
    int32_t LedMode = 0;

    // 检查客户端和设备是否为空
    if(client == NULL || client->device == NULL){
        HDF_LOGE("driver device is NULL");
        return HDF_ERR_INVALID_OBJECT;
    }
```

```
12
13          // 检查 LED 配置版本是否支持
14          if(g_LedCfg.led_version != LED_VERSION){
15              HDF_LOGE("led version is not support");
16              return HDF_FAILURE;
17          }
18
19          switch(cmdId){
20              case LED_WRITE:
21                  // 从数据缓冲区读取 LED 模式
22                  result = HdfSbufReadInt32(dataBuf,&LedMode);
23                  if(result ){
24                      // 根据 LED 模式控制 GPIO
25                      LedGpioCtl(g_LedCfg.led_number, (LedMode == LED_ON) ? LED_ON: LED_OFF);
26                  }
27                  break;
28              default:
29                  // 不支持的命令 ID
30                  HDF_LOGE("cmdId is not support");
31                  break;
32          }
33          return result;
34
35      }
```

LedGpioCtl：控制指定 GPIO（LED）的高低电平，从而控制 LED 灯的开关。

- 参数：gpio（GPIO 号），mode（LED 模式，开或关）。
- 流程：设置 GPIO 为输出方向，根据 mode 设置 GPIO 的电平，最后记录日志。

```
1       static int32_t LedGpioCtl(uint16_t gpio, uint32_t mode){
2           // 设置 GPIO 电平为高电平
3           uint16_t level = GPIO_VAL_HIGH;
4
5           // 设置 GPIO 为输出方向
6           if(HDF_SUCCESS != GpioSetDir(gpio, GPIO_DIR_OUT)){
7               // 设置 GPIO 方向失败
8               HDF_LOGE("GpioSetDir fail");
9               return HDF_FAILURE;
10          }
11
12          // 根据 mode 设置 GPIO 电平
13          if(mode == LED_ON){
14              level = GPIO_VAL_HIGH;
15          }else if(mode==LED_OFF){
16              level = GPIO_VAL_LOW;
```

```
        }

        // 日志记录 GPIO 操作
        HDF_LOGE("%s:Write gpio %d:%d",__func__,gpio,mode);

        // 向 GPIO 写入电平
        if(HDF_SUCCESS != GpioWrite(gpio, level)){
            // 写入 GPIO 电平失败
            HDF_LOGE("GpioWrite fail",__func__);
        }

        return HDF_SUCCESS;
    }
```

3.4.4 添加内核编译

编译内核时将该 HDF 驱动编译到镜像中，接下来编写驱动编译脚本 Makefile，代码如下所示：

```
include drivers/hdf/khdf/platform/platform.mk
obj-y += led_driver.o
```

加入编译体系，填加模块目录到 drivers/hdf_core/adapter/khdf/linux/Makefile 文件

```
obj-$(CONFIG_DRIVERS_HDF) += ../../../../../topeet/demos/hdf_led/driver/
```

3.4.5 编写应用 APP

在应用代码中我们实现如下功能：

当应用程序启动后会获取命令行参数。如果命令行没有参数，LED 灯将循环闪烁；如果命令行带有参数，则根据传输的参数控制 LED 灯的开启或关闭。通过 HdfIoServiceBind 绑定 LED 灯的 HDF 服务，获取 HDF 空间缓存区，并向该缓冲区写入控制数据。然后，通过 LED_WRITE 命令将数据发送到 HDF 驱动，从而控制 LED 灯的亮灭。在程序结束时，会回收 HDF 空间缓冲区和 HDF 服务。

接下来编写应用测试文件 led_test.c，完整代码如下所示。

```
#include "stdio.h"
#include "stdlib.h"
#include "unistd.h"
#include "hdf_base.h"
#include "hdf_io_service.h"
#include "hilog/log.h"
```

```
7
8       #undef LOG_TAG
9       #undef LOG_DOMAIN
10      #define LOG_TAG "led_test"
11      #define LOG_DOMAIN 0xD0020240
12
13      #define ARGS_NUM 2
14
15      #define LED_SERVICE_NAME "topeet_led_service"
16      #define LED_WRITE 1
17
18      /**
19       * @brief 主函数，用于控制 LED 灯的开关状态
20       *
21       * 根据传入的参数控制 LED 灯的开关状态，如果没有传入参数，则进入主循环，不断切换 LED
22  灯的开关状态。
23       *
24       * @param argc 命令行参数的数量
25       * @param argv 命令行参数的数组
26       *
27       * @return 返回 HDF_SUCCESS 表示成功，否则返回错误码
28       */
29      int main(int argc, char *argv[]){
30          int ret = HDF_SUCCESS;
31          int32_t mode = -1;
32
33          // 判断命令行参数数量
34          if (argc == ARGS_NUM) {
35              // 将命令行参数转换为整数并赋值给 mode
36              mode = atoi(argv[1]);
37              // 打印 mode 的状态
38              printf("mode[%s][0x%x]\n",(mode==1)?"On":"Off",mode);
39
40          } else {
41              // 命令行参数数量不正确，打印提示信息
42              printf("led main loop. \n");
43          }
44
45          // 绑定 LED 服务
46          struct HdfIoService *serv = HdfIoServiceBind(LED_SERVICE_NAME);
47          if(serv == NULL){
48              // 绑定服务失败，打印错误信息并返回 -1
49              HILOG_ERROR(LOG_APP, "get service %s failed!", LED_SERVICE_NAME);
50              return -1;
```

```
51          }
52          // 打印绑定服务成功的日志
53          HILOG_ERROR(LOG_APP, "get service %s succeed", LED_SERVICE_NAME);
54
55          // 获取默认大小的 SBuf 对象
56          struct HdfSBuf *data = HdfSbufObtainDefaultSize();
57          if(data == NULL){
58              // 获取 SBuf 对象失败，打印错误信息并返回 -1
59              HILOG_ERROR(LOG_APP,"obtain data failed\n");
60              return -1;
61          }
62          // 打印获取 SBuf 对象成功的日志
63          HILOG_ERROR(LOG_APP,"obtain data succeed\n");
64
65          // 如果 mode 为 -1，则进入循环
66          if(mode == -1){
67              while(1){
68                  // 清空 SBuf 对象
69                  HdfSbufFlush(data);
70                  // 向 SBuf 对象写入整数 1
71                  if(!HdfSbufWriteInt32(data, 1)){
72                      // 写入数据失败，打印错误信息并返回 -1
73                      HILOG_ERROR(LOG_APP,"write data failed");
74                      return -1;
75                  }
76                  // 调用 Dispatch 方法，发送 LED 写入命令
77                  ret = serv->dispatcher->Dispatch(&serv->object, LED_WRITE, data, NULL);
78                  usleep(500 * 1000);
79
80                  // 清空 SBuf 对象
81                  HdfSbufFlush(data);
82                  // 向 SBuf 对象写入整数 0
83                  if(!HdfSbufWriteInt32(data, 0)){
84                      // 写入数据失败，打印错误信息并返回 -1
85                      HILOG_ERROR(LOG_APP,"write data failed");
86                      return -1;
87                  }
88                  // 调用 Dispatch 方法，发送 LED 写入命令
89                  ret = serv->dispatcher->Dispatch(&serv->object, LED_WRITE, data, NULL);
90                  usleep(500 * 1000);
91              }
92
93          } else {
94              // 如果 mode 不为 -1，则向 SBuf 对象写入 mode 值
```

```
95              if(!HdfSbufWriteInt32(data, mode)){
96                  // 写入数据失败，打印错误信息并返回 -1
97                  HILOG_ERROR(LOG_APP,"write data failed");
98                  return -1;
99              }
100             // 调用 Dispatch 方法，发送 LED 写入命令
101             ret = serv->dispatcher->Dispatch(&serv->object, LED_WRITE, data, NULL);
102             // 打印 Dispatch 成功的日志
103             HILOG_ERROR(LOG_APP,"Dispatch succeed");
104         }
105
106         // 回收 SBuf 对象
107         HdfSbufRecycle(data);
108         // 回收服务对象
109         HdfIoServiceRecycle(serv);
110         // 打印主程序退出的日志
111         HILOG_INFO(LOG_APP,"[%s] main exit.",LOG_TAG);
112
113         return ret;
114     }
```

接下来编写应用 APP 的 GN 文件 BUILD.gn，代码内容如下所示：

```
1       HDF_FRAMEWORKS = "//drivers/hdf_core/framework"
2       HDF_ADAPTER = "//drivers/hdf_core/adapter"
3
4       import("//build/ohos.gni")
5       import("$HDF_ADAPTER/uhdf2/uhdf.gni")
6
7       print("demos: compile led_test")
8
9       ohos_executable("led_test"){
10          sources = ["led_test.c"]
11          include_dirs = [
12              "$HDF_FRAMEWORKS/include",
13              "$HDF_FRAMEWORKS/include/core",
14              "$HDF_FRAMEWORKS/include/osal",
15              "$HDF_FRAMEWORKS/include/platform",
16              "$HDF_FRAMEWORKS/include/utils",
17              "$HDF_ADAPTER/uhdf2/ipc/include",
18              "$HDF_ADAPTER/uhdf2/osal/include",
19              "//base/hiviewdfx/hilog/interfaces/native/innerkits/include",
20              "//third_party/bounds_checking_function/include",
21          ]
22          external_deps = [
```

```
23                "c_utils:utils",
24                "hdf_core:libhdf_platform",
25                "hdf_core:libhdf_utils",
26                "hilog:libhilog",
27
28            ]
29
30            cflags = [
31                "-Wall",
32                "-Wextra",
33                "-Werror",
34                "-Wno-format",
35                "-Wno-format-extra-args",
36            ]
37            part_name = "demos"
38            install_enable = true
39
40        }
```

上面的代码用于构建一个“led_test”的可执行文件的构建脚本，它使用了 GN（Generate Ninja）构建系统，这是一种元构建系统，用于生成 Ninja 构建文件。

- 1-2 行定义了两个变量 HDF_FRAMEWORKS 和 HDF_ADAPTER，它们分别指向 HDF（Hardware Driver Foundation，硬件驱动框架）核心框架和适配器的路径。这些路径是相对于项目根目录的。
- 4-5 行 使用 import 语句导入两个 GNI（GN Include）文件。GNI 文件是 GN 构建系统用来包含变量定义、函数和模板的文件。这里导入的文件可能包含了一些预定义的变量、函数或构建规则，用于支持构建过程。//build/ohos.gni 可能包含了 OpenHarmony 特有的构建配置，而$HDF_ADAPTER/uhdf2/uhdf.gni 可能包含了与 uHDF（Unified Hardware Driver Framework，统一硬件驱动框架）相关的配置。
- 7 行 打印一条消息到控制台，表明正在编译 led_test 示例。
- 9-40 行 定义一个名为 led_test 的 ohos_executable 目标，这是一个构建规则，用于生成一个可执行文件。下面是该目标的具体配置：

  sources：指定源文件列表，这里只有一个文件 led_test.c。

  include_dirs：指定头文件搜索路径列表。这些路径用于在编译时查找包含的文件（#include 指令引用的文件）。这些路径包括了 HDF 框架、适配器的多个子目录，以及一些第三方库和内部工具库的头文件路径。

external_deps：指定外部依赖项列表。这些依赖项是在构建过程中需要链接的库。这里列出了几个库，如 c_utils:utils、hdf_core:libhdf_platform 等，这些库提供了构建 led_test 所需的功能。

cflags：指定传递给 C 编译器的标志列表。这里包括了一些常见的编译选项，如-Wall（打开所有警告）、-Wextra（打开额外警告）、-Werror（将所有警告视为错误）、以及两个用于关闭特定警告的选项。

part_name：指定构建产物所属的部件名称，这里是 demos。

install_enable：设置为 true，表示构建产物应该被安装。这可能意味着在构建成功后，led_test 可执行文件会被复制到某个特定的目录，以便于执行或分发。

3.5 在产品中新增子系统

在 build/subsystem_config.json 文件中增加名为 topeet 的子系统，在 3.4 节已经新建了 topeet 文件夹存放子系统代码。添加 topeet 子系统进行一个登记，说明路径和子系统名称，如下所示：

```
“topeet”: {
    “path”: “topeet”,
    “name”: ”topeet”
}
```

```
222    "llvmproject": {
223      "path": "toolchain/llvm-project",
224      "name": "llvmproject"
225    },
226    "topeet":{
227      "path":"topeet",
228      "name":"topeet"
229    }
230  }
```

在 vendor/hihope/rk3568/config.json 文件中增加 topeet 子系统的引入，如下所示：

```
{
        "subsystem": "topeet",
        "components": [
          {
            "component": "demos",
            "features": [
            ]
          }
        ]
```

```
}
```

```
        },
        {
          "component": "vulkan-headers",
          "features": []
        }
      ]
    },
    {
      "subsystem": "topeet",
      "components": [
        {
          "component": "demos",
          "features": [
          ]
        }
      ]

    }
  ]
}
```

修改完成之后，保存修改。

3.6 编译源码

重新编译 Openharmony4.1 源码，如下所示：

```
./build.sh --product-name rk3568 --ccache
```

或者单独编译部件

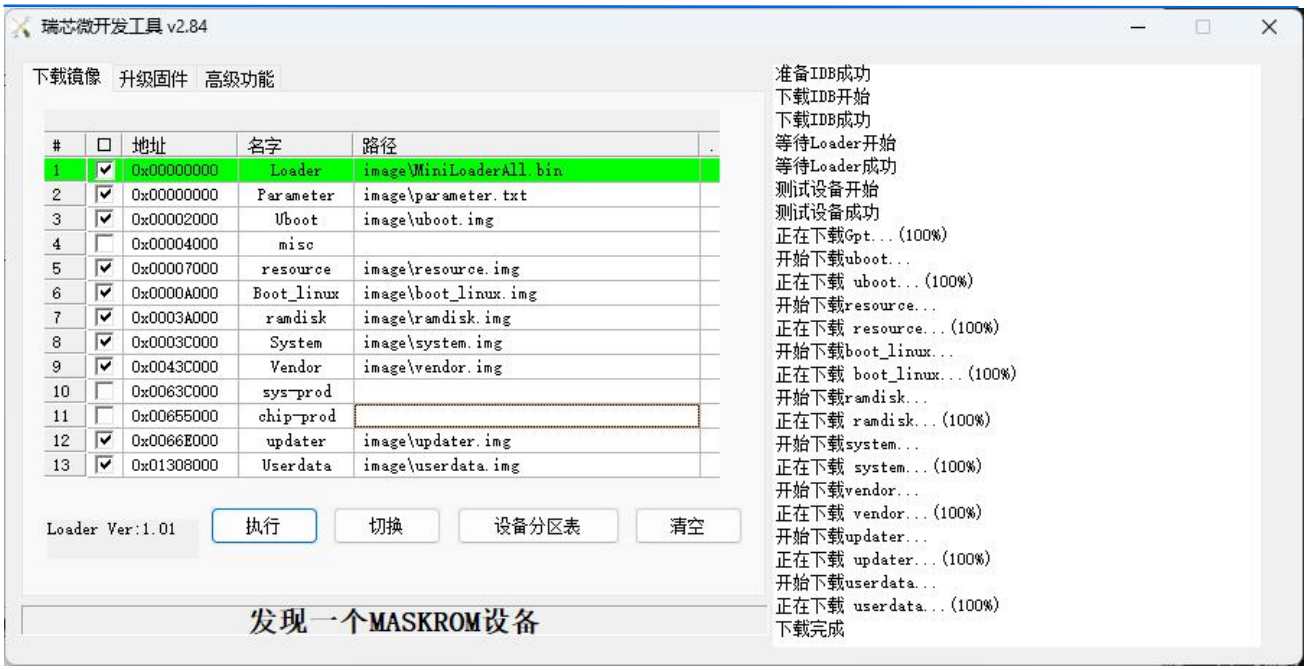
```
./build.sh --product-name rk3568 --build-target demos --ccache
```

编译之后，在源码 out/rk3568/topeet 目录下生成编译产物，如下图所示：

```
topeet@ubuntu:~/OH4.1/OpenHarmony-v4.1-Release/OpenHarmony/out/rk3568/topeet$ tree
.
└── demos
    └── led_test

1 directory, 1 file
topeet@ubuntu:~/OH4.1/OpenHarmony-v4.1-Release/OpenHarmony/out/rk3568/topeet$
```

3.7 LED 测试

将编译好的镜像全部进行烧写，镜像在源码根目录 out/rk3568/packages/phone/images/目录下。

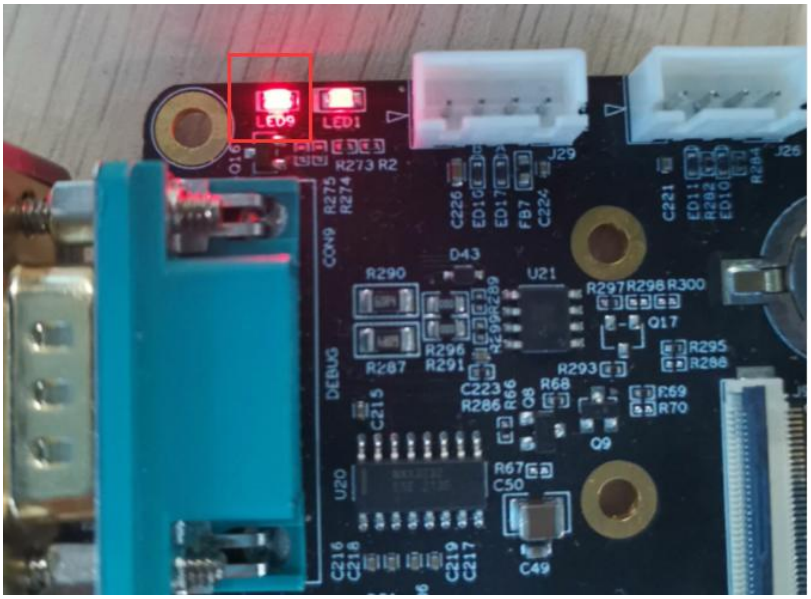


烧写完成之后，在调试串口查看打印日志，如下图所示：

```
4.788361]
4.788510] [I/device_node] launch devnode HDF_PLATFORM_PWM_0
4.788533] [I/pwm_adapter] HdfPwmInit: entry!
4.789087] [I/osal_cdev] add cdev HDF_PLATFORM_PWM_0 success
4.789087]
4.789272] [I/device_node] launch devnode topeet_led_service
4.789301] [I/led_driver] g_LedDriverEntry: HdfLedDriverBind success NodeName[topeet_led_config]
4.789327] [I/led_driver] g_LedDriverEntry: HdfLedDriverInit success
4.789802] [I/osal_cdev] add cdev topeet_led_service success
4.789802]
```

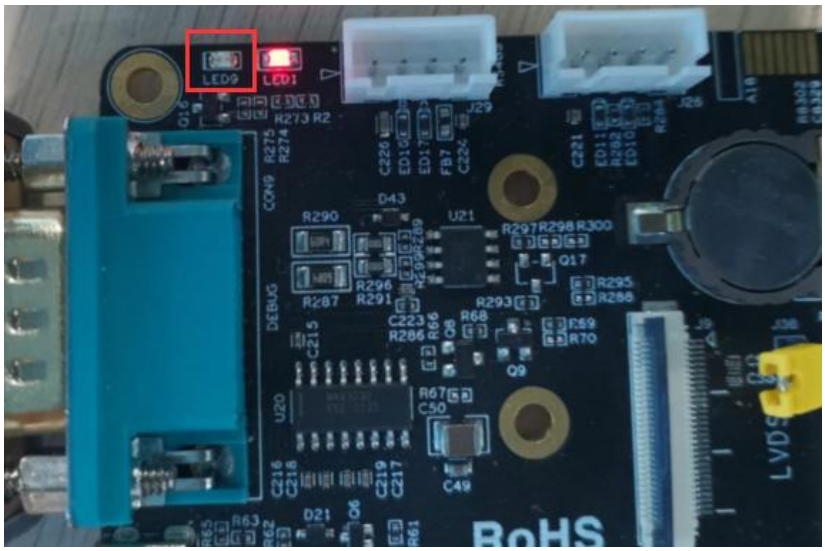
然后打开 hdc 工具，运行测试程序，输入“led_test 1”，LED 灯点亮，如下图所示：

```
D:\Cadence\SPB_Data>hdc.exe shell
# led_test 1
mode[On][0x1]
#
```



输入“led_test 0”,LED 灯熄灭，如下图所示：

```
#
# led_test 0
mode[Off][0x0]
#
```



输入“led_test”，LED 灯闪烁。

至此，LED 灯实验完结，撒花~

实战：第二篇 Openharmony 配置 HDF 控制 UART

第 4 章 UART 基础知识

4.1 什么是串口

串口（Serial Port）也叫串行通信接口，通常也叫做 COM 接口，是通用串行数据总线，用于异步通信。该总线双向通信，可以实现全双工传输。

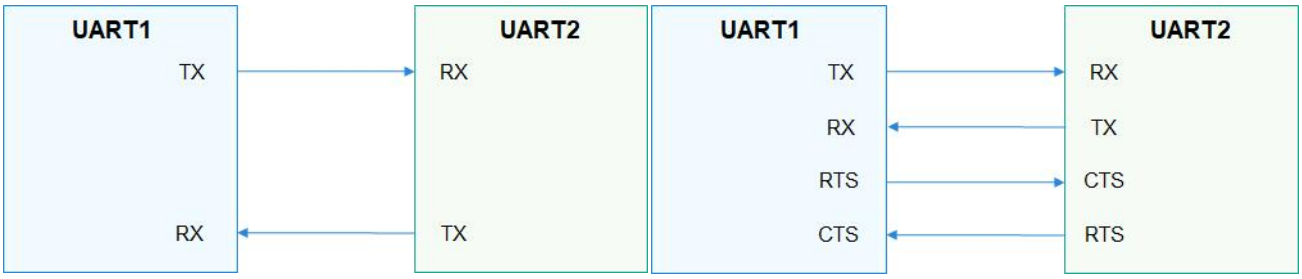
两个 UART 设备的连接示意图如下，UART 与其他模块一般用 2 线（图 1）或 4 线（图 2）相连，它们分别是：

TX: 发送数据端，和对端的 RX 相连

RX：接收数据端，和对端的 TX 相连

RTS: 发送请求信号，用于指示本设备是否准备好，可接受数据，和对端 CTS 相连

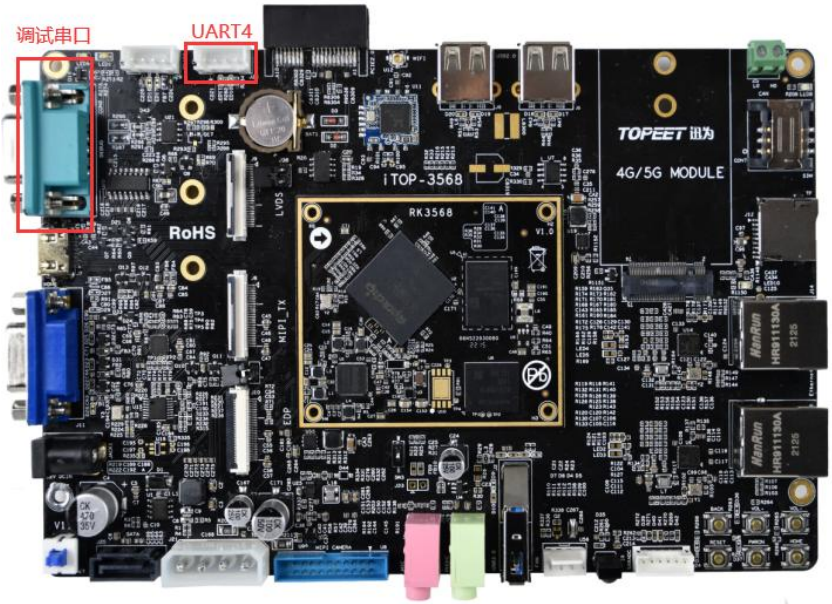
CTS: 允许发送信号，用于判断是否可以向对端发送数据，和对端 RTS 相连



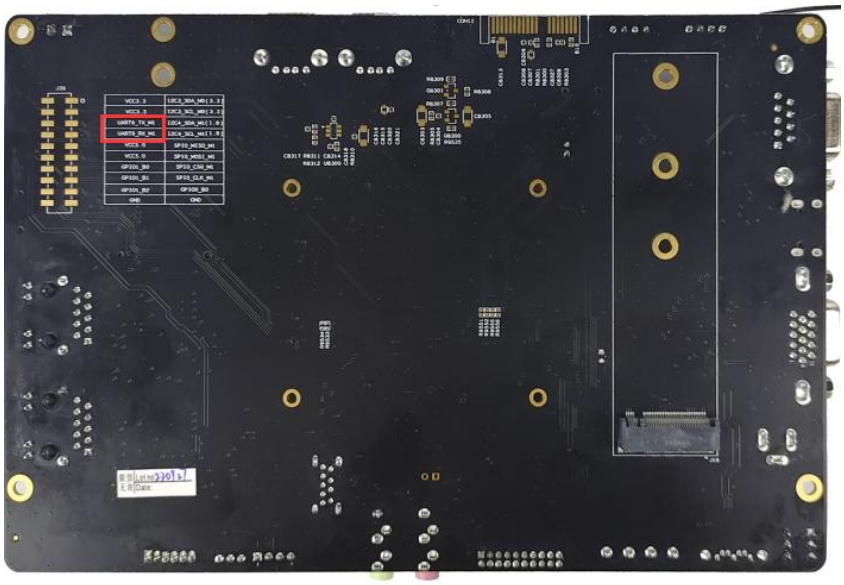
串口通信之前，收发双方需要约定好一些参数：波特率，数据格式（起始位，数据位，校验位，停止位）等。通信过程中，串口通过 TX 发送给对端数据，通过 RX 接收对端发送的数据。当串口接收缓存达到预定的门限值时，RTS 变为不可发送数据，对端的 CTS 检测到不可发送数据，则停止发送数据。

4.2 开发板串口简介

串口在实际工作中会经常使用，比如在嵌入式 Linux 下使用调试串口，它用来登录 Linux 系统，输出日志。串口也会和外部的一些模块通信，比如 GPS 模块，RS485 模块等。根据电平的不同，串口分为 TTL 电平串口和 RS232 电平串口。在迅为 iTOP-RK3568 开发板上引出了三路串口，如下图所示，绿色的 db9 座子为 RS232 电平调试串口，UART4 为 TTL 电平串口。



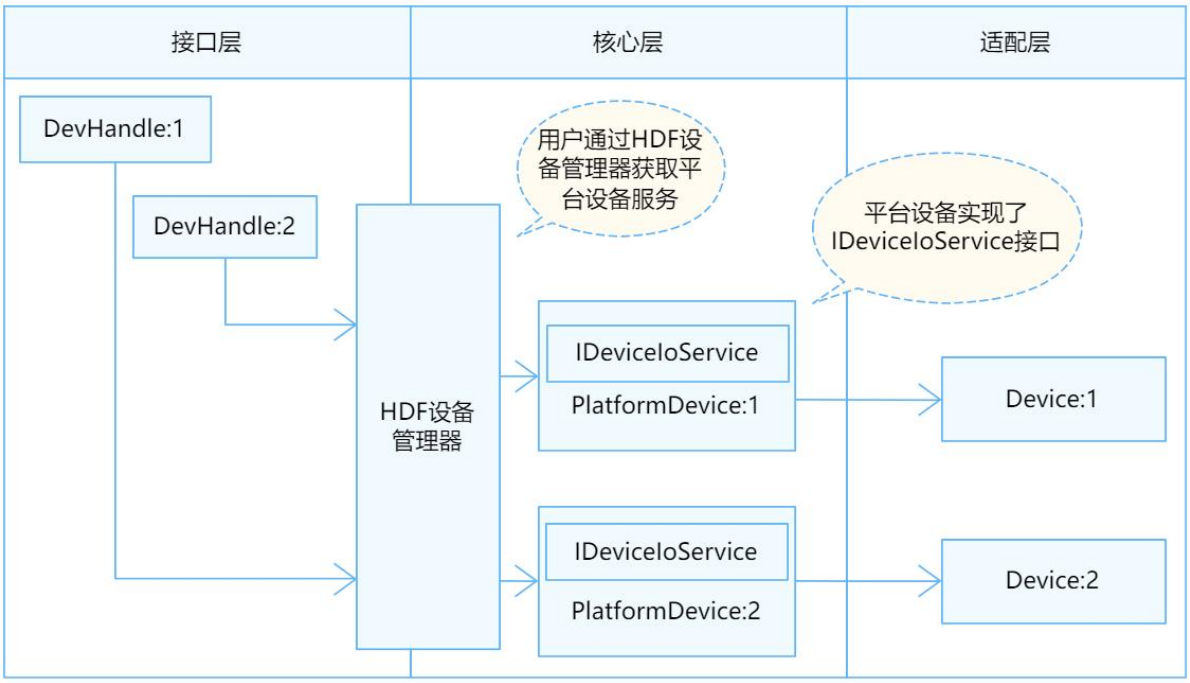
在开发板背面的 20pin 座子上有一组 TTL 电平串口——串口 9。



在接下来的实验中，我们将配置三路串口——串口 4/7/9。

第 5 章 UART 接口运作机制

在 HDF 框架中，串口接口适配模式采用独立服务模式，如下图所示：



如上图所示，每个设备对象会独立发布一个设备服务来处理外部访问，设备管理器接收到 API 的访问请求之后，通过提取该请求的参数，达到调用实际设备对象的相应内部方法的目的。这种模式可以直接借助 HDF 设备管理器的服务管理能力，但需要为每个设备单独配置设备节点，增加内存占用。

UART 模块各分层的作用如下所述：

- 接口层提供打开 UART 设备，UART 设备读取指定长度数据，UART 设备写入指定长度数据，设置 UART 设备波特率，获取设 UART 设备波特率，设置 UART 设备属性，获取 UART 设备波特率，设置 UART 设备传输模式，关闭 UART 设备的接口。
- 核心层主要提供 UART 控制器的创建，移除以及管理的能力，通过钩子函数与适配层交互。
- 适配层主要是将钩子函数的功能实例化，实现具体的功能。

第 6 章 实操-HDF 驱动配置 UART

6.1 修改 HCS 配置

对于不同的平台，需要在对应的平台目录修改对应的 hcs 文件，接下来示例为在 rk3568 下新增 uart4 uart9 uart7 的修改方法。

修改 vendor/hihope/rk3568/hdf_config/khdf/device_info/device_info.hcs 文件，device_info.hcs 中添加以下内容：

```
1    device3 :: deviceNode {
2        policy = 2;
3        permission = 0644;
```

```
            priority = 40;
            moduleName = "HDF_PLATFORM_UART";
            serviceName = "HDF_PLATFORM_UART_4";
            deviceMatchAttr = "rockchip_rk3568_uart_4";
        }
        device4 :: deviceNode {
            policy = 2;
            permission = 0644;
            priority = 40;
            moduleName = "HDF_PLATFORM_UART";
            serviceName = "HDF_PLATFORM_UART_7";
            deviceMatchAttr = "rockchip_rk3568_uart_7";
        }
        device5 :: deviceNode {
            policy = 2;
            permission = 0644;
            priority = 40;
            moduleName = "HDF_PLATFORM_UART";
            serviceName = "HDF_PLATFORM_UART_9";
            deviceMatchAttr = "rockchip_rk3568_uart_9";
        }
```

在配置过程中要注意以下几点：

1 device3，device4，device5 是自定义的，可以根据实际情况修改

2 policy 表示服务策略，取值为 0 时，表示不发布服务，取值为 1 时表示向内核态发布服务，取值为 2 时表示向内核用户态发布服务

3 moduleName 的值要与驱动实现的 HdfDriverEntry 结构体中的 moduleName 相同。

4 deviceMatchAttr 的值表示驱动的私有配置信息

5 serviceName 表示服务名称，服务加载成功之后会在开发板的/dev/目录下生成节点。例如 HDF_PLATFORM_UART_9 后面跟着的数据 9 是 UartOpen()的端口号

6.2 配置 rk3568_uart_config.hcs

修改 vendor/hihope/rk3568/hdf_config/khdf/platform/rk3568_uart_config.hcs 文件，添加如下内容，如下所示：

```
        device_uart_0x0004 :: uart_device {
            num = 4;
            match_attr = "rockchip_rk3568_uart_4";
        }
        device_uart_0x0007 :: uart_device {
            num = 7;
            match_attr = "rockchip_rk3568_uart_7";
        }
        device_uart_0x0009 :: uart_device {
            num = 9;
```

```
        match_attr = "rockchip_rk3568_uart_9";
    }
```

在上面的配置中需要注意以下几点：
1 device_uart_0x0004 中的后缀“0x0004”是串口编号。
2 num 与 driver_name 值“ttyS”组成驱动设备名，例如 ttyS4。UartOpen 函数参数 port,则表示上述 uart 设备排列序号，比如 num=4 的 UartOpen 函数 port=4。
3 match_attr 的名称必须是 rockchip_rk3568_uart_x,和 device_info.hcs 中要写一样。

6.3 Openharmony UART 平台驱动

在 drivers/hdf_core/adapter/khdf/linux/platform/uart/uart_adapter.c 中编写了对接 Linux UART 驱动的相关代码，这部分不需要大家进行修改，感兴趣的话可以自己研究下 uart_adapter.c 文件。

6.4 UART 应用开发

6.4.1 UART 驱动 API 接口介绍

UART 驱动 API 接口如下所示，具体的 API 详见 drivers/hdf_core/framework/include/platform/uart_if.h 文件。

接口名	接口描述
DevHandle UartOpen(uint32_t port)	UART 获取设备句柄
void UartClose(DevHandle handle)	UART 释放设备句柄
int32_t UartRead(DevHandle handle, uint8_t *data, uint32_t size)	从 UART 设备中读取指定长度的数据
int32_t UartWrite(DevHandle handle, uint8_t *data, uint32_t size)	向 UART 设备中写入指定长度的数据
int32_t UartGetBaud(DevHandle handle, uint32_t *baudRate)	UART 获取波特率
int32_t UartSetBaud(DevHandle handle, uint32_t baudRate)	UART 设置波特率
int32_t UartGetAttribute(DevHandle handle, struct UartAttribute *attribute)	UART 获取设备属性
int32_t UartSetAttribute(DevHandle handle, struct UartAttribute *attribute)	UART 设置设备属性
int32_t UartSetTransMode(DevHandle handle, enum UartTransMode mode)	UART 设置传输模式

(1) UartOpen
在使用 UART 进行通信时，首先要调用 UartOpen 获取 UART 设备句柄，该函数会返回指定端口号的 UART 设备句柄。函数原型如下所示：

```
DevHandle UartOpen(uint32_t port);
```

其中，参数 port 是 UART 设备号。UartOpen 返回值为 NULL 表示获取 UART 设备句柄失败，正常情况下返回 UART 设备句柄。
假设系统重的 UART 端口号为 4，获取该 UART 设备句柄的示例如下所示：

```
    DevHandle handle = NULL;    // UART 设备句柄
```

```
    uint32_t port = 4;              // UART 设备端口号

    handle = UartOpen(port);
    if (handle == NULL) {
        HDF_LOGE("UartOpen: open uart_%u failed!\n", port);
        return;
    }
```

(2) UartSetBaud

在通信之前，需要设置 UART 的波特率，函数原型如下所示：

```
int32_t UartSetBaud(DevHandle handle, uint32_t baudRate);
```

其中，参数 handle 表示 UART 设备句柄，baudRate 表示待设置的波特率值。UartSetBaud 返回值为 HDF_SUCCESS 表示波特率设置成功，返回值为负数表示 UART 设置波特率失败。

(3) UartGetBaud

设置 UART 的波特率后，可以通过获取波特率接口来查看 UART 当前的波特率。函数原型如下所示：

```
int32_t UartGetBaud(DevHandle handle, uint32_t *baudRate);
```

其中，参数 handle 表示 UART 设备句柄，baudRate 表示待设置的波特率值。UartSetBaud 返回值为 HDF_SUCCESS 表示获取波特率成功，返回值为负数表示 UART 获取波特率失败。

(4) UartSetAttribute

在通信之前，需要设置 UART 的设备属性。函数原型如下所示：

```
int32_t UartSetAttribute(DevHandle handle, struct UartAttribute *attribute);
```

其中，handle 表示 UARt 设备句柄，attribute 表示待设置的设备属性。UartSetAttribute 返回值为 HDF_SUCCESS 表示 UART 设置属性成功，返回值为负数表示 UART 设置设备属性失败。

(5) UartGetAttribute

设置 UART 的设备属性后，可以通过获取设备属性接口来查看 UART 当前的设备属性。函数原型如下所示：

```
int32_t UartGetAttribute(DevHandle handle, struct UartAttribute *attribute);
```

其中，handle 表示 UART 设备句柄，attribute 表示接收 UART 设备属性的指针。UartGetAttribute 返回值为 HDF_SUCCESS 表示 UART 获取属性成功，返回值为负数表示 UART 获取设备属性失败。

(6) UartSetTransMode

在通信之前，需要设置 UART 的传输模式。函数原型如下所示：

```
int32_t UartSetTransMode(DevHandle handle, enum UartTransMode mode);
```

其中，handle 表示 UART 设备句柄，mode 表示待设置的传输模式。UartSetTransMode 返回值为 HDF_SUCCESS 表示 UART 设置传输模式成功，返回值返回负数表示 UART 设置传输模式失败。

(7) UartWrite

向 UART 设备写入指定长度的数据。函数原型如下所示：

```
int32_t UartWrite(DevHandle handle, uint8_t *data, uint32_t size);
```

其中，handle 表示 UART 设备句柄，data 表示待写入数据的指针，size 表示待写入数据的长度。UartWrite 返回值为 HDF_SUCCESS 表示 UART 写数据成功，返回值为负数表示 UART 写数据失败。

(8) UartRead

从 UART 设备中读取指定长度的数据，函数原型如下所示：

```
int32_t UartRead(DevHandle handle, uint8_t *data, uint32_t size);
```

其中，参数 handle 表示 UART 设备句柄，data 表示接收读取数据的指针，size 表示待读取数据的长度。UartRead 返回值为非负数表示 UART 读取到的数据长度，返回值为负数，表示 UART 读取数据失败。

(9) UartClose

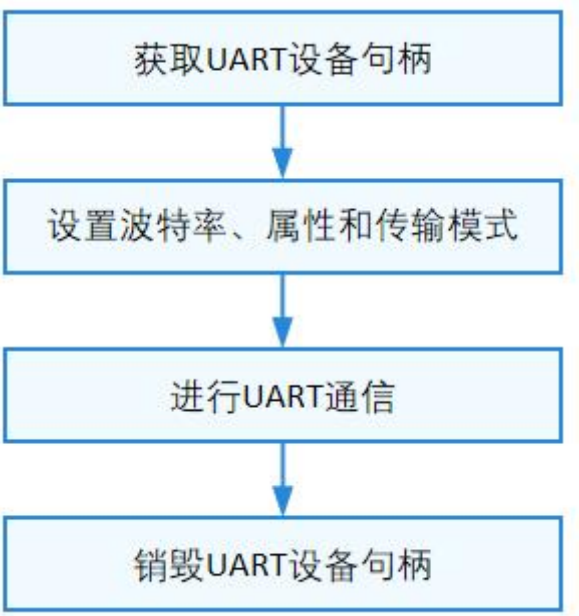
UART 通信完成之后，需要销毁 UART 设备句柄，函数原型如下所示：

```
void UartClose(DevHandle handle);
```

其中，参数 handle 表示 UART 设备句柄。

6.4.2 编写应用测试 APP

使用 UART 的一般流程如下所示：



接下来编写应用测试文件 uart_test.c，完整代码如下所示：

```
#include "stdio.h"
#include "stdlib.h"
#include "unistd.h"
#include "hdf_base.h"
#include "hdf_io_service.h"
#include "hilog/log.h"
#include "uart_if.h"   // 假设这个头文件包含了所有 UART 相关函数的声明和宏定义

#define STRING_MAXSIZE 100
#define HDF_SUCCESS 0

// 假设这些宏在 uart_if.h 中定义
#ifndef PRINT_ERROR
#define PRINT_ERROR(...) fprintf(stderr, __VA_ARGS__)
#endif

#ifndef PRINT_INFO
#define PRINT_INFO(...) printf(__VA_ARGS__)
#endif

int main(int argc, char* argv[])
{
    DevHandle handle = NULL;
    struct UartAttribute attribute;
    int32_t ret = 0;
    uint8_t wbuff[STRING_MAXSIZE] = "HelloWorld";
    uint8_t rbuff[STRING_MAXSIZE] = {0};   // 初始化接收缓冲区为 0
```

```
28          uint32_t m_uart_port = atoi(argv[1]);   // UART 端口的标识符
29          uint32_t m_uart_baudrate = 115200;       // UART 的波特率
30
31          // 初始化 UART 属性
32          attribute.dataBits = UART_ATTR_DATABIT_8;     // UART 传输数据位宽，一次传输 8 个 bit
33          attribute.parity = UART_ATTR_PARITY_NONE;     // UART 传输数据无校检
34          attribute.stopBits = UART_ATTR_STOPBIT_1;     // UART 传输数据停止位为 1 位
35          attribute.rts = UART_ATTR_RTS_DIS;            // UART 禁用 RTS
36          attribute.cts = UART_ATTR_CTS_DIS;            // UART 禁用 CTS
37          attribute.fifoRxEn = UART_ATTR_RX_FIFO_EN;    // UART 使能 RX FIFO
38          attribute.fifoTxEn = UART_ATTR_TX_FIFO_EN;    // UART 使能 TX FIFO
39
40          // 打开 UART 设备
41          handle = UartOpen(m_uart_port);
42          if (handle == NULL) {
43              PRINT_ERROR("UartOpen: open uart port %u failed!\n", m_uart_port);
44              return -1;
45          }
46          PRINT_INFO("UartOpen successful and uart port = %u\n", m_uart_port);
47
48          // 设置 UART 波特率
49          ret = UartSetBaud(handle, m_uart_baudrate);
50          if (ret != HDF_SUCCESS) {
51              PRINT_ERROR("UartSetBaud: set baud failed, ret %d\n", ret);
52              goto ERR;
53          }
54          PRINT_INFO("UartSetBaud successful and uart baudrate = %u\n", m_uart_baudrate);
55
56          // 设置 UART 设备属性
57          ret = UartSetAttribute(handle, &attribute);
58          if (ret != HDF_SUCCESS) {
59              PRINT_ERROR("UartSetAttribute: set attribute failed, ret %d\n", ret);
60              goto ERR;
61          }
62          PRINT_INFO("UartSetAttribute successful\n");
63
64          // 获取 UART 设备属性（可选）
65          ret = UartGetAttribute(handle, &attribute);
66          if (ret != HDF_SUCCESS) {
67              PRINT_ERROR("UartGetAttribute: get attribute failed, ret %d\n", ret);
68              goto ERR;
69          }
70          PRINT_INFO("UartGetAttribute successful\n");
71
```

```
72          // 设置 UART 传输模式为阻塞模式
73          ret = UartSetTransMode(handle, UART_MODE_RD_BLOCK);
74          if (ret != HDF_SUCCESS) {
75              PRINT_ERROR("UartSetTransMode: set trans mode failed, ret %d\n", ret);
76              goto ERR;
77          }
78          PRINT_INFO("UartSetTransMode successful\n");
79
80          // 向 UART 设备写入数据
81          ret = UartWrite(handle, wbuff, (uint32_t)strlen((char *)wbuff));
82          if (ret != HDF_SUCCESS) {
83              PRINT_ERROR("UartWrite: write data failed, ret %d\n", ret);
84              goto ERR;
85          }
86          PRINT_INFO("UartWrite successful and wbuff = %s\n", wbuff);
87
88          // 从 UART 设备读取数据
89          do{
90          ret = UartRead(handle, rbuff, STRING_MAXSIZE);
91          if (ret < 0) {
92              PRINT_ERROR("UartRead: read data failed, ret %d\n", ret);
93              goto ERR;
94          }
95          sleep(1);
96          } while (!ret);
97          rbuff[ret] = '\0'; // 确保字符串以 null 结尾（如果读取的是文本数据）
98          PRINT_INFO("UartRead successful and rbuff = %s\n", rbuff);
99
100     ERR:
101         // 关闭 UART 设备句柄（确保在 handle 非空时关闭）
102         if (handle != NULL) {
103             UartClose(handle);
104         }
105         return ret;
106 }
```

接下来编写应用 APP 的 GN 文件 BUILD.gn，代码内容如下所示：

```
1       HDF_FRAMEWORKS = "//drivers/hdf_core/framework"
2       HDF_ADAPTER = "//drivers/hdf_core/adapter"
3
4       import("//build/ohos.gni")
5       import("$HDF_ADAPTER/uhdf2/uhdf.gni")
6
7       print("demos: compile uart_test")
```

```
8
9       ohos_executable("uart_test"){
10          sources = ["uart_test.c"]
11          include_dirs = [
12              "$HDF_FRAMEWORKS/include",
13              "$HDF_FRAMEWORKS/include/core",
14              "$HDF_FRAMEWORKS/include/osal",
15              "$HDF_FRAMEWORKS/include/platform",
16              "$HDF_FRAMEWORKS/include/utils",
17              "$HDF_ADAPTER/uhdf2/ipc/include",
18              "$HDF_ADAPTER/uhdf2/osal/include",
19              "//base/hiviewdfx/hilog/interfaces/native/innerkits/include",
20              "//third_party/bounds_checking_function/include",
21          ]
22          external_deps = [
23              "c_utils:utils",
24              "hdf_core:libhdf_platform",
25              "hdf_core:libhdf_utils",
26              "hilog:libhilog",
27
28          ]
29
30          cflags = [
31              "-Wall",
32              "-Wextra",
33              "-Werror",
34              "-Wno-format",
35              "-Wno-format-extra-args",
36          ]
37          part_name = "demos"
38          install_enable = true
39      }
```

6.5 编译源码

重新编译 Openharmony4.1 源码，如下所示：

```
./build.sh --product-name rk3568 --ccache
```

或者单独编译部件

```
./build.sh --product-name rk3568 --build-target demos --ccache
```

编译之后，在源码 out/rk3568/topeet 目录下生成编译产物，如下图所示：

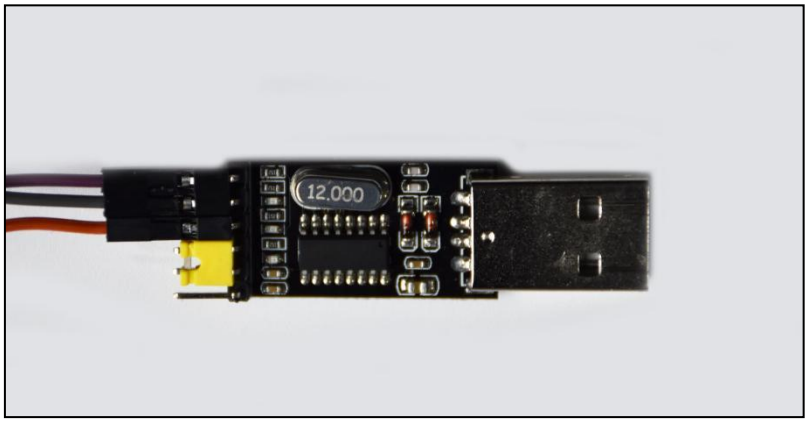
```
topeet@ubuntu:~/OH4.1/OpenHarmony-v4.1-Release/OpenHarmony/out/rk3568/topeet$ tree
.
└── demos
    ├── led_test
    └── uart_test

1 directory, 2 files
topeet@ubuntu:~/OH4.1/OpenHarmony-v4.1-Release/OpenHarmony/out/rk3568/topeet$
topeet@ubuntu:~/OH4.1/OpenHarmony-v4.1-Release/OpenHarmony/out/rk3568/topeet$
```

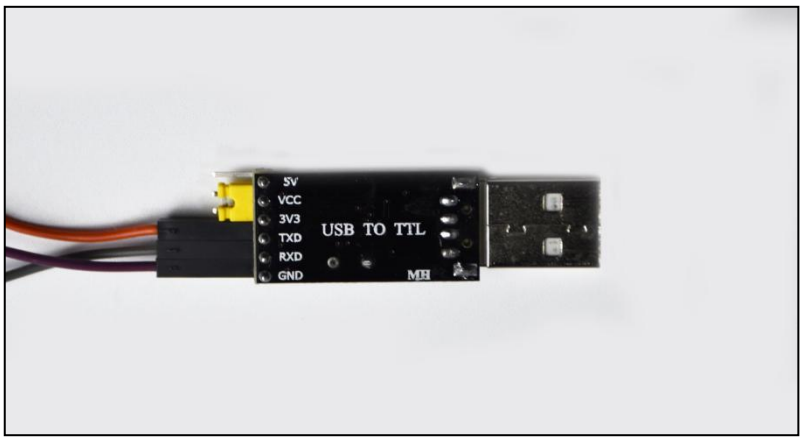
6.6 UART 测试

将编译好的镜像全部进行烧写，镜像在源码根目录 out/rk3568/packages/phone/images/目录下。

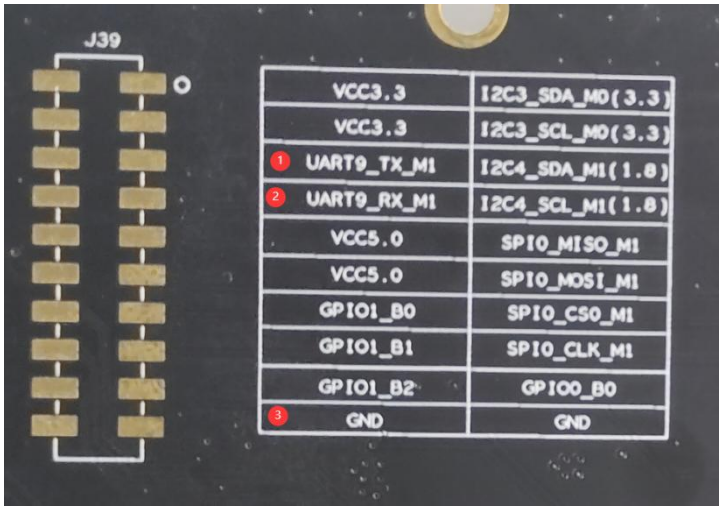
烧写完成之后，连接串口工具，本小节将要测试串口 9，对应的设备节点是 /dev/HDF_PLATFORM_UART_9。

作者使用 usb 转 TTL（需要自行准备）来进行测试，如下图所示：





串口 9 在开发板底板的背面，如下图所示，USB 转 TTL 的 RXD 要使用导线连接到开发板串口 9 的 UART9_TX_M1,USB 转 TTL 的 TXD 要使用导线连接到开发板串口 9 的 UART9_RX_M1,USB 转 TTL 的地接到 GND。



在电脑上打开串口助手，选择串口 4 对应的串口号和波特率，注意：默认波特率为 115200！打开串口，如下图所示：

输入以下命令运行测试程序发送数据和接收数据，发送的数据为 112233445678，数据信息可以自定义。

```
uart_test 9
```

```
#
# uart_test 9
UartOpen successful and uart port = 9
UartSetBaud successful and uart baudrate = 115200
UartSetAttribute successful
UartGetAttribute successful
UartSetTransMode successful
UartWrite successful and wbuff = HelloWorld
```

串口软件接收到字符串“HelloWorld”

然后在串口软件发送数据“112233445678”，如下图所示：

串口终端收到发送的字符串，如下图所示：

```
# uart_test 9
UartOpen successful and uart port = 9
UartSetBaud successful and uart baudrate = 115200
UartSetAttribute successful
UartGetAttribute successful
UartSetTransMode successful
UartWrite successful and wbuff = HelloWorld
UartRead successful and rbuff = 112233445678


#
```

至此，串口实验完结。